

Unidade II

3 APRENDIZADO SUPERVISIONADO

O aprendizado supervisionado é uma das técnicas mais utilizadas na área de machine learning, um dos principais campos da inteligência artificial. Ele permite que sistemas "aprendam" a partir de dados rotulados, ou seja, exemplos previamente conhecidos com suas respectivas respostas corretas. Essa abordagem é muito eficaz para realizar previsões, classificações e detecções automáticas em diferentes domínios.

No aprendizado supervisionado, o modelo é treinado com um conjunto de dados de entrada e suas respectivas saídas desejadas (rótulos). O objetivo é fazer com que o sistema aprenda uma função de mapeamento entre entradas e saídas, de modo que ele consiga generalizar esse conhecimento para prever resultados de novos dados.

Quadro 5 – Exemplo simples

Entrada (X) – Área da casa	Saída (Y) – Preço
50 m ²	R\$ 200.000
80 m ²	R\$ 300.000
100 m ²	R\$ 400.000

O modelo aprende a prever o preço da casa (Y) com base em sua área (X).

O processo de aprendizado supervisionado envolve três etapas principais:

- **Treinamento:** o modelo recebe os dados de entrada com suas saídas conhecidas e ajusta seus parâmetros internos para aprender o padrão.
- **Validação:** o desempenho é testado em um conjunto separado de dados (validação), para evitar sobreajuste (overfitting).
- **Teste/predict:** o modelo é aplicado a novos dados para fazer previsões com base no que aprendeu.

Tipos de problemas supervisionados

Existem dois principais tipos de problemas no aprendizado supervisionado:

- **Classificação:** o objetivo é atribuir uma categoria (classe) à entrada. Exemplos:

- Classificar um e-mail como "spam" ou "não spam".
- Identificar se uma imagem contém um "gato" ou "cachorro".
- Prever se um cliente irá "comprar" ou "não comprar".

Saída: categórica (exemplos: sim/não, A/B/C).

- **Regressão:** o objetivo é prever um valor numérico contínuo com base nos dados de entrada. Exemplos:

- Prever o valor de um imóvel.
- Estimar a temperatura para os próximos dias.
- Calcular a nota final de um aluno com base em suas atividades.

Saída: contínua (exemplos: R\$ 350.000, 27 °C, 8.5).

Quadro 6 – Algoritmos comuns de aprendizado supervisionado

Algoritmo	Tipo	Aplicação comum
Regressão linear	Regressão	Previsão de preços, vendas
Regressão logística	Classificação	Diagnóstico médico (doença: sim/não)
Árvores de decisão	Ambos	Classificação de clientes
K-Nearest Neighbors (k-NN)	Ambos	Reconhecimento de padrões
Máquinas de vetores de suporte (SVM)	Ambos	Classificação de imagens, textos
Redes neurais artificiais	Ambos	Reconhecimento facial, voz, escrita

Aplicações reais

O aprendizado supervisionado é utilizado em sistemas reais. Veja alguns exemplos:

- **Saúde:** diagnóstico de doenças a partir de exames (detectar câncer em imagens).
- **Finanças:** previsão de inadimplência com base em perfil de crédito.
- **Marketing:** prever churn (abandono de clientes).
- **Indústria:** prever falhas de máquinas com base em dados de sensores.
- **Educação:** prever desempenho acadêmico dos alunos.

Quadro 7 – Vantagens e desvantagens

Vantagens	Desvantagens
Resultados geralmente precisos	Requer grande quantidade de dados rotulados
Facilidade de interpretar o processo	Risco de sobreajuste se o modelo for muito complexo
Aplicável a muitos domínios	Difícil generalizar com dados de baixa qualidade

O sucesso de um modelo supervisionado depende da qualidade dos dados. Para isso, é importante:

- ter rótulos confiáveis;
- normalizar os dados numéricos;
- remover valores ausentes ou inconsistentes;
- separar os dados em conjuntos de treinamento, validação e teste.

O aprendizado supervisionado é uma das abordagens mais eficazes e utilizadas na inteligência artificial. Ele permite que sistemas sejam treinados com exemplos concretos e se tornem capazes de realizar previsões e classificações automáticas em contextos reais. Para profissionais da área de computação, compreender seus fundamentos é essencial para aplicar IA em projetos de software inteligente, sistemas preditivos e automações inteligentes.

3.1 Classificação e regressão

Ambas compartilham o mesmo princípio: treinar um modelo com dados de entrada rotulados para que ele consiga prever saídas a partir de novos dados. No entanto, o tipo de saída esperada define se o problema é de classificação ou de regressão.

3.1.1 Classificação

A classificação é o processo de atribuir uma etiqueta (classe) a um dado de entrada. O objetivo é prever a qual categoria uma nova instância pertence, com base no aprendizado anterior.

A figura 16 a seguir representa um exemplo de classificação: de um lado estão classificadas as bolinhas azuis; do outro, as bolinhas vermelhas. Assim, temos dois objetos separados, cada um com sua classificação. Por exemplo, as bolinhas vermelhas poderiam ser pessoas doentes e as bolinhas azuis, pessoas saudáveis.

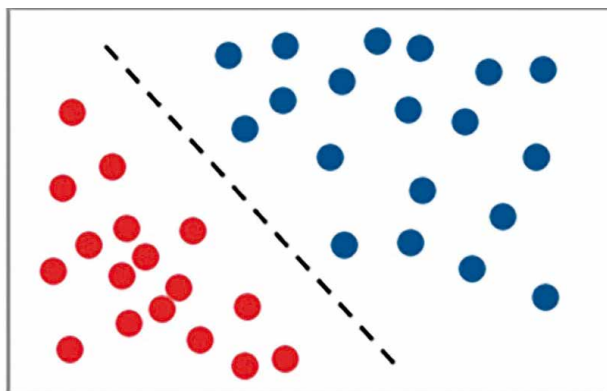


Figura 16 – Exemplo de classificação

Exemplo clássico

Um dos exemplos mais populares é um sistema de filtragem de e-mails que classifica as mensagens recebidas como "spam" ou "não spam". O modelo é treinado com milhares de e-mails previamente categorizados, aprendendo padrões que indicam características típicas de spam (como palavras-chave, remetentes suspeitos etc.) e de e-mails legítimos. Após o treinamento, o sistema pode aplicar esse conhecimento para classificar automaticamente novos e-mails.

Outros exemplos práticos de classificação

- **Diagnóstico médico:** a partir de sintomas, exames e histórico do paciente, o modelo classifica se uma determinada doença está presente ou ausente. Por exemplo, prever se um paciente tem ou não diabetes.
- **Reconhecimento facial:** dada uma imagem de um rosto, o sistema identifica a que pessoa ela pertence, comparando com um banco de rostos conhecidos. Cada pessoa representa uma classe distinta.
- **Análise de sentimentos em redes sociais:** um modelo analisa comentários ou postagens e os classifica conforme o sentimento predominante: positivo, negativo ou neutro.
- **Classificação de espécies:** dado um conjunto de características (formato das orelhas, tipo de pelo e tamanho), um modelo pode classificar um animal como gato, cachorro, pássaro, entre outros.

Tipo de saída

A característica principal de problemas de classificação é que a saída do modelo é categórica ou discreta. Isso significa que as respostas possíveis são pertencentes a conjuntos limitados de categorias ou rótulos, como:

- **Binária:** sim/não, verdadeiro/falso, aprovado/reprovado.

- **Multiclasse:** classe A, classe B, classe C...
- **Multirrótulo (em casos mais complexos):** uma mesma entrada pode pertencer a mais de uma categoria simultaneamente.

3.1.2 Regressão

A regressão trata de prever um valor contínuo com base em entradas numéricas. O modelo aprende uma relação matemática entre os dados de entrada e um resultado numérico.

Exemplo clássico

Previsão do valor de um imóvel com base em características como:

- área construída (em metros quadrados);
- localização geográfica;
- número de quartos e banheiros;
- idade do imóvel;
- presença de garagem ou outras comodidades.

O modelo é treinado com dados históricos de imóveis cujas características e valores de venda são conhecidos. Após o treinamento, ele pode prever o preço estimado de um novo imóvel com base nas suas características.

Outros exemplos práticos de regressão

- **Previsão de temperatura:** dado um conjunto de variáveis climáticas (umidade, pressão atmosférica, estação do ano, horário do dia), o modelo estima a temperatura em graus Celsius ou Fahrenheit.
- **Estimativa de vendas mensais:** utilizando dados como histórico de vendas, campanhas promocionais, sazonalidade e tendências do mercado, o modelo prevê quantas unidades de um produto serão vendidas no próximo mês.
- **Previsão da nota de um aluno:** com base no desempenho anterior do aluno, participação nas aulas, frequência e entregas de atividades, o modelo pode prever a nota final de uma disciplina.
- **Estimativa de consumo de energia:** a partir de variáveis como hora do dia, tipo de equipamento, ocupação de um ambiente, temperatura externa, entre outros, é possível prever o consumo elétrico em kWh.

Tipo de saída

A principal característica dos problemas de regressão é que a variável alvo (saída) é numérica e contínua. Ou seja, o valor previsto pode assumir qualquer número dentro de um intervalo, com ou sem casas decimais. Exemplos de saídas típicas:

- **Temperatura:** 25,7 °C.
- **Preço:** R\$ 1.250,00.
- **Nota:** 8,9.
- **Consumo:** 432,75 kWh.

3.1.3 Comparação entre classificação e regressão

No aprendizado supervisionado, tanto a classificação quanto a regressão são tarefas importantes, porém com objetivos distintos e aplicabilidades específicas. A seguir, apresentamos uma comparação entre essas duas abordagens:

Tipo de saída

A principal diferença entre os dois está no tipo de saída que produzem. A classificação lida com variáveis categóricas ou discretas, ou seja, o modelo atribui a entrada a uma classe específica, como "spam" ou "não spam". Já a regressão trabalha com valores numéricos contínuos, prevendo uma quantidade específica, como o preço de um imóvel ou a temperatura de uma cidade.

Objetivo

Na classificação, o objetivo é identificar a qual categoria ou classe uma determinada entrada pertence, com base em exemplos rotulados previamente. Na regressão, o foco é estimar um valor numérico que representa uma variável de interesse.

Avaliação do modelo

Como os objetivos são diferentes, as métricas de avaliação também variam. Modelos de classificação são comumente avaliados por meio de acurácia, precisão, revocação e F1-score, que medem a capacidade do modelo de identificar corretamente as categorias.

A acurácia é a métrica mais simples e direta. Ela mede a proporção de predições corretas em relação ao total de predições. É calculada da seguinte forma:

$$\text{Acurácia} = \frac{\text{Número de predições corretas}}{\text{Total de predições}}$$

A acurácia é útil quando as classes estão balanceadas, isto é, há aproximadamente a mesma quantidade de exemplos de cada classe. No entanto, em situações de desequilíbrio de classes, a acurácia pode ser enganosa, pois o modelo pode ficar viciado para prever a classe majoritária.

A precisão, por sua vez, é a proporção de exemplos verdadeiramente positivos (verdadeiros positivos) em relação a todos os exemplos classificados como positivos (verdadeiros positivos mais falsos positivos). É uma métrica que avalia a qualidade das previsões positivas do modelo.

$$\text{Precisão} = \frac{\text{Verdadeiros positivos}}{\text{Verdadeiros positivos} + \text{Falsos positivos}}$$

A precisão é útil quando o foco está em minimizar os falsos positivos, ou seja, quando prever incorretamente a classe positiva é problemático.

Já o recall (sensibilidade ou taxa de verdadeiros positivos) é a proporção de exemplos verdadeiramente positivos (verdadeiros positivos) em relação a todos os exemplos que realmente pertencem à classe positiva (verdadeiros positivos mais falsos negativos). Essa métrica é importante quando o foco está em capturar a maioria dos verdadeiros positivos.

$$\text{Recall} = \frac{\text{Verdadeiros positivos}}{\text{Verdadeiros positivos} + \text{Falsos negativos}}$$

Por fim, o F1-score é uma métrica que combina precisão e recall em uma única medida. É a média harmônica entre essas duas métricas e é especialmente útil quando você deseja encontrar um equilíbrio entre precisão e recall.

$$\text{F1 - Score} = 2 * \frac{\text{Precisão} * \text{Recall}}{\text{Precisão} + \text{Recall}}$$

O F1-score é útil em situações de desequilíbrio de classes, em que é preciso levar em consideração tanto os falsos positivos quanto os falsos negativos.

A acurácia é uma métrica geral que pode ser enganosa em problemas de desequilíbrio de classes, enquanto a precisão, recall e F1-score oferecem insights mais detalhados sobre o desempenho do modelo em diferentes aspectos das previsões de classificação. A escolha da métrica depende do contexto e das prioridades do problema enfrentado.

Os modelos de regressão são avaliados com base em métricas de erro, por exemplo, o erro absoluto médio (MAE), o erro quadrático médio (MSE) ou o erro percentual absoluto médio (Mape), que indicam o quão distante o valor previsto está do valor real.

O MAE mede a média das diferenças absolutas entre os valores reais e os valores previstos.

Interpretação: quanto menor o MAE, melhor o modelo.

Fórmula:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

É fácil de interpretar, pois mantém as unidades da variável original.

O MSE, por sua vez, calcula a média dos quadrados das diferenças entre os valores reais e os previstos.

Interpretação: penaliza mais fortemente erros grandes (por isso é sensível a outliers).

Fórmula:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Por fim, o Mape expressa o erro absoluto médio em termos percentuais.

Interpretação: mostra, em média, qual a porcentagem de erro do modelo em relação aos valores reais.

Fórmula:

$$MAPE = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|$$

Ele é útil para entender a precisão em termos relativos, mas não é ideal quando há valores reais próximos de zero.

Visualização dos resultados

Os resultados de classificação geralmente são representados por gráficos de barras (mostrando a frequência ou distribuição das classes), enquanto os de regressão são melhor visualizados com gráficos de linha ou dispersão, que mostram a relação entre as variáveis e os valores numéricos previstos.

Exemplo comparativo

Um mesmo conjunto de dados pode ser utilizado para diferentes finalidades dentro do aprendizado supervisionado, dependendo do objetivo da análise. Imagine que temos informações sobre veículos, como a quilometragem rodada, o ano de fabricação, a marca e o tipo de combustível.

Se a meta for prever o valor de mercado do carro em reais, ou seja, estimar um número contínuo com base nas características fornecidas, estamos diante de um problema de regressão. O modelo aprenderá, a partir de exemplos anteriores, a estimar preços com base nos atributos dos veículos.

Por outro lado, se o objetivo for classificar os veículos em categorias como "econômico" ou "não econômico", com base nas mesmas variáveis de entrada, estamos tratando de um problema de classificação. Nesse caso, o modelo aprenderá a associar padrões dos dados a rótulos fixos e discretos, definindo a qual categoria cada veículo pertence.

Esse exemplo mostra como a natureza do problema – prever um valor contínuo ou atribuir uma categoria – define se o modelo será de regressão ou de classificação, ainda que utilize o mesmo conjunto de dados.

A escolha entre aplicar um modelo de classificação ou de regressão depende diretamente do tipo de saída que se deseja prever. Quando o objetivo é estimar valores contínuos, como preço, temperatura ou consumo, utiliza-se regressão. Já quando se busca identificar categorias específicas, como níveis de risco, tipos de clientes ou diagnósticos médicos, opta-se por classificação.

Em algumas situações, é possível transformar um problema de regressão em um problema de classificação, por meio da categorização dos valores numéricos. Por exemplo, em vez de prever exatamente o preço de um produto, pode-se classificá-lo em faixas como "baixo", "médio" e "alto valor", convertendo o problema para um formato categórico.

Além disso, a qualidade dos dados de entrada e a escolha adequada dos algoritmos exercem grande influência nos resultados obtidos. Dados inconsistentes, incompletos ou irrelevantes podem comprometer o desempenho do modelo, assim como a seleção de algoritmos mal ajustados ao tipo de problema ou ao volume de dados pode limitar sua precisão e aplicabilidade. Portanto, uma análise criteriosa desses fatores é essencial para o sucesso de qualquer aplicação de aprendizado supervisionado.

Classificação e regressão são, portanto, os dois principais tipos de problemas do aprendizado supervisionado. Compreender a diferença entre eles é essencial para escolher a abordagem certa em projetos de IA. Classificação é utilizada quando se deseja atribuir categorias, enquanto regressão é usada para estimar valores numéricos. Profissionais da área de desenvolvimento de sistemas inteligentes devem ser capazes de reconhecer o tipo de problema, escolher o modelo adequado e avaliar corretamente os resultados obtidos.

3.2 Algoritmos de aprendizado supervisionado

No aprendizado supervisionado, escolher o algoritmo correto é fundamental para obter bons resultados em tarefas de previsão ou classificação. Entre os algoritmos mais populares e utilizados estão a regressão linear, a regressão logística e o K-Nearest Neighbors (k-NN).

3.2.1 Regressão linear

A regressão linear é um algoritmo utilizado para resolver problemas de regressão, ou seja, aqueles em que se deseja prever valores numéricos contínuos.

Seu funcionamento consiste em tentar encontrar a melhor reta (função linear) que se ajusta aos dados, modelando a relação entre uma ou mais variáveis independentes (entradas) e uma variável dependente (saída).

Equação básica:

$$y = ax + b$$

Onde:

- y é a saída prevista (exemplo: resultado de um teste);
- x é a entrada (exemplo: o número de horas estudadas);
- a é o coeficiente angular (peso);
- b é o intercepto.

Utilizando a Python para exemplificar, consideremos um conjunto de dados com duas variáveis: o número de horas estudadas (X) e o resultado de um teste (Y). Usaremos a regressão linear para tentar prever os resultados dos testes com base nas horas estudadas. Assim, começamos importando as bibliotecas necessárias e criando alguns dados fictícios.

O código a seguir (figura 17) cria um gráfico (figura 18) com os pontos representando os dados reais, a linha de regressão linear e alguns pontos verdes que indicam as previsões para novas horas estudadas. A regressão linear é usada para encontrar a linha que melhor se ajusta aos dados, minimizando a soma dos erros quadráticos.

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # Dados fictícios de horas estudadas e resultados do teste
5  horas_estudadas = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
6  resultados_teste = np.array([35, 47, 60, 72, 80, 92, 100, 112, 120, 130])
7
8  # Calculando a média das horas estudadas e dos resultados do teste
9  media_horas = np.mean(horas_estudadas)
10 media_resultados = np.mean(resultados_teste)
11
12 # Calculando os coeficientes da regressão
13 numerator = np.sum((horas_estudadas - media_horas) * (resultados_teste - media_resultados))
14 denominator = np.sum((horas_estudadas - media_horas) ** 2)
15 slope = numerator / denominator
16 intercept = media_resultados - slope * media_horas
17
18 # Função de regressão linear
19 def regressao_linear(x):
20     return slope * x + intercept
21
22 # Fazendo a previsão para algumas horas estudadas
23 horas_para_previsao = np.array([11, 12, 13, 14])
24 previsao_resultados = regressao_linear(horas_para_previsao)
25
26 # Plotando os dados e a linha de regressão
27 plt.scatter(horas_estudadas, resultados_teste, label="Dados reais")
28 plt.plot(horas_estudadas, regressao_linear(horas_estudadas), color='red', label="Regressão linear")
29 plt.scatter(horas_para_previsao, previsao_resultados, color='green', label="Previsão")
30 plt.xlabel("Horas Estudadas")
31 plt.ylabel("Resultados do Teste")
32 plt.legend()
33 plt.show()

```

Figura 17

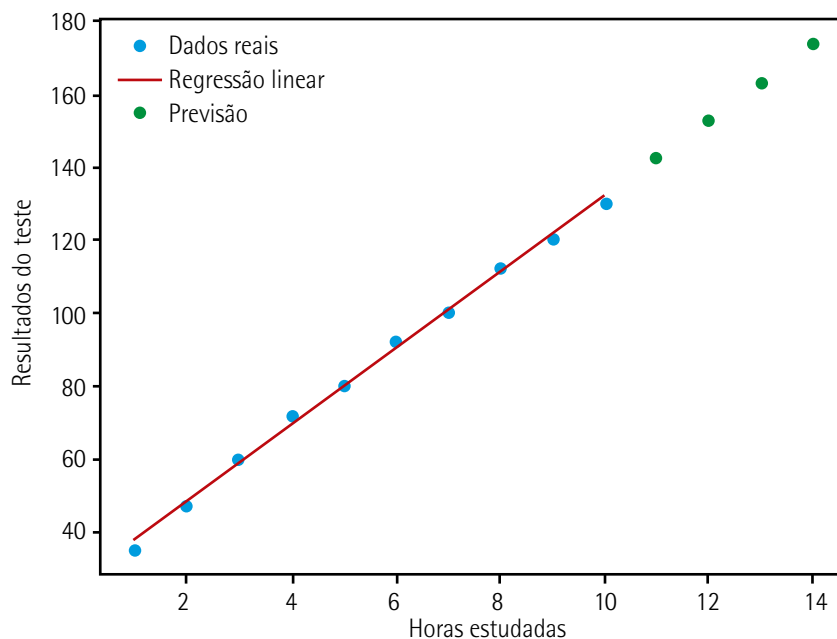


Figura 18



Lembrete

Considerando que esse é um exemplo simples, a qualidade da previsão pode variar dependendo dos dados reais e do modelo estatístico utilizado.

Um dos principais atrativos de modelos simples, como a regressão linear, está na sua facilidade de compreensão e implementação. Trata-se de uma técnica intuitiva, muito utilizada e de rápida execução, sendo eficaz quando há uma relação linear clara entre as variáveis independentes e a variável alvo. Além disso, o tempo necessário para o treinamento do modelo é geralmente muito curto, o que o torna vantajoso em contextos que exigem respostas rápidas com baixo custo computacional.

No entanto, esse tipo de modelo apresenta algumas limitações importantes. Ele tende a apresentar um mal desempenho quando os dados apresentam padrões não lineares, ou seja, quando a relação entre as variáveis não pode ser representada por uma linha reta. Outra desvantagem relevante é sua sensibilidade a outliers, que são valores extremos capazes de distorcer significativamente os resultados do modelo, comprometendo a precisão das previsões. Por isso, seu uso deve ser bem avaliado de acordo com as características dos dados e da complexidade do problema.

3.2.2 Regressão logística

A regressão logística é usada para classificação binária, isto é, quando a saída desejada possui apenas duas classes (exemplos: sim/não, verdadeiro/falso).

Apesar do nome, a saída da regressão logística é probabilística. Esse modelo prevê a probabilidade de um exemplo pertencer à classe 1 e utiliza a função sigmoide para transformar a saída em um valor entre 0 e 1.

Função sigmoide:

$$f(x) = 1 / (1 + e^{(-z)})$$

Onde z é o resultado de uma equação linear ($ax + b$).

Imagine que temos um conjunto de dados que contém informações sobre alunos e queremos prever se um aluno será aprovado (1) ou reprovado (0) com base em suas horas de estudo e notas em exames anteriores. Usaremos a regressão logística para construir um modelo que faça essa previsão.

Consideremos as figuras 19 e 20 a seguir.

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from sklearn.model_selection import train_test_split
4  from sklearn.linear_model import LogisticRegression
5  from sklearn.metrics import accuracy_score, classification_report
6
7  # Dados de exemplo
8  horas_de_estudo = np.array([5, 1, 6, 7, 4, 9, 2, 8, 3, 10])
9  notas_exame = np.array([85, 50, 80, 90, 60, 95, 55, 92, 70, 98])
10 aprovado = np.array([1, 0, 1, 1, 0, 1, 0, 1, 0, 1]) # 1: aprovado, 0: reprovado
11
12 # Matriz de características (X) e vetor de rótulos (y)
13 X = np.column_stack((horas_de_estudo, notas_exame))
14 y = aprovado
15
16 # Divisão dos dados (80% treino, 20% teste)
17 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
18
19 # Criação do modelo de regressão logística
20 modelo = LogisticRegression()
21
22 # Treinamento do modelo
23 modelo.fit(X_train, y_train)
24
25 # Previsões com os dados de teste
26 previsoes = modelo.predict(X_test)
27
28 # Probabilidades estimadas de aprovação
29 probs = modelo.predict_proba(X_test)
30
31 # Avaliação do desempenho
32 acuracia = accuracy_score(y_test, previsoes)
33 print("Acurácia do modelo:", acuracia)
34 print("\nRelatório de Classificação:\n", classification_report(y_test, previsoes))
35
36 # Visualização dos dados e da decisão do modelo
37 plt.figure(figsize=(10, 6))
38
39 # Cores para os pontos
40 cores = ['red' if label == 0 else 'green' for label in y]
41
42 # Dados reais
43 plt.scatter(X[:, 0], X[:, 1], c=cores, edgecolors='k', s=100, label='Dados Reais (aprovado/reprovado)')
44 plt.xlabel('Horas de Estudo')
45 plt.ylabel('Nota no Exame')
46 plt.title('Regressão Logística: Previsão de Aprovação')
47 plt.grid(True)
48
49 # Criar uma malha de pontos para desenhar a fronteira de decisão
50 x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
51 y_min, y_max = X[:, 1].min() - 5, X[:, 1].max() + 5
52 xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100), np.linspace(y_min, y_max, 100))
53 malha = np.c_[xx.ravel(), yy.ravel()]
54 Z = modelo.predict(malha).reshape(xx.shape)
55
56 # Desenhar a fronteira de decisão
57 plt.contourf(xx, yy, Z, alpha=0.2, cmap=plt.cm.Paired)
58 plt.legend()
59 plt.show()

```

Figura 19

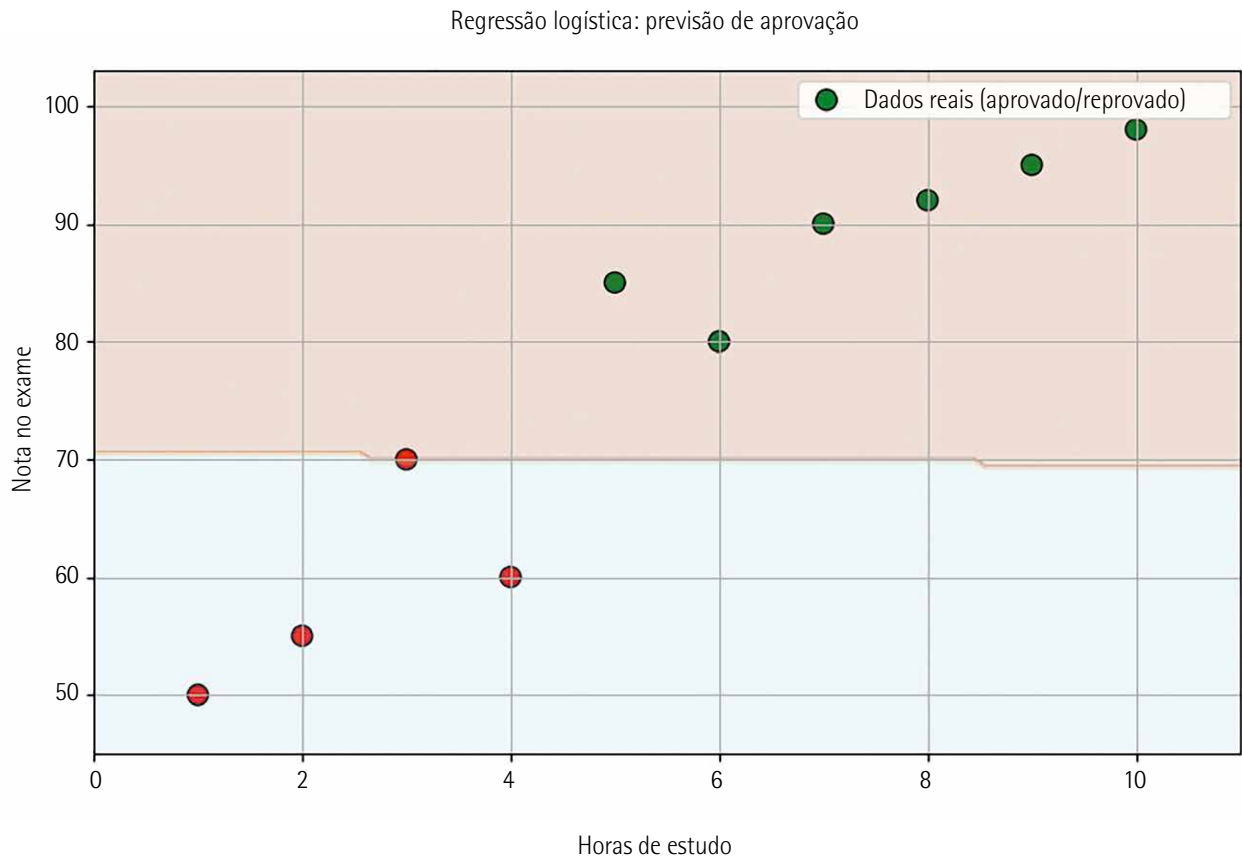


Figura 20

Nesse exemplo, o modelo foi treinado com os dados de treinamento e, em seguida, avaliado usando os dados de teste para calcular a acurácia do modelo.

A regressão logística é um método poderoso para a análise de classificação binária, permitindo estimar a probabilidade de um evento ocorrer com base em um conjunto de variáveis independentes. Ela é utilizada na prática de ciência de dados e fornece insights valiosos para tomada de decisões.

A regressão logística apresenta diversas vantagens, especialmente quando aplicada a problemas de classificação binária simples. Uma de suas principais qualidades é a facilidade de interpretação estatística, já que os coeficientes do modelo indicam diretamente o impacto de cada variável na probabilidade de ocorrência de um determinado evento. Além disso, ela funciona bem quando há uma separação linear entre as classes, sendo uma opção eficiente e de baixo custo computacional para problemas com essa característica.

No entanto, o modelo também possui limitações importantes. Por padrão, ele não lida bem com problemas que envolvem múltiplas classes, a menos que sejam aplicadas estratégias específicas, como a abordagem um contra todos (one-vs-rest). Outro ponto a considerar é que seu desempenho pode ser limitado em situações com dados complexos ou com padrões não lineares, em que modelos mais avançados, como árvores de decisão, redes neurais ou algoritmos baseados em ensemble, tendem a oferecer melhores resultados.

3.2.3 K-Nearest Neighbors (k-NN)

O k-NN é um algoritmo baseado em instâncias e pode ser usado tanto para classificação quanto para regressão.

Para prever a saída de um novo dado, o k-NN compara sua distância com os exemplos do conjunto de treinamento e seleciona os ks vizinhos mais próximos. A previsão é baseada nesses vizinhos.

- Para **classificação**, a classe mais comum entre os vizinhos é escolhida.
- Para **regressão**, é feita a média dos valores dos vizinhos.
- A distância usada, geralmente, é a **distância euclidiana**, mas outras métricas podem ser aplicadas.

Vamos exemplificar a aplicabilidade do algoritmo k-NN com a base de dados Iris, usando a biblioteca scikit-learn em Python.

O código carrega o conjunto de dados Iris (figura 21), divide-o em conjuntos de treinamento e teste, cria um classificador k-NN usando a classe KNeighborsClassifier do scikit-learn e treina o classificador com o conjunto de treinamento, especificando que queremos usar três vizinhos mais próximos (você pode ajustar o valor de k para experimentar diferentes números de vizinhos). Em seguida, avalia a precisão do classificador no conjunto de teste (figura 22).

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from sklearn.datasets import load_iris
4  from sklearn.model_selection import train_test_split
5  from sklearn.neighbors import KNeighborsClassifier
6  from sklearn.metrics import accuracy_score, classification_report
7
8  # 1. Carregando o conjunto de dados Iris
9  iris = load_iris()
10 X = iris.data          # Matrizes de características (comprimento e largura das pétalas e sépalas)
11 y = iris.target        # Vetor com os rótulos das classes (0, 1, 2)
12
13 # 2. Dividindo os dados em treino e teste (80% treino, 20% teste)
14 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
15
16 # 3. Criando o classificador KNN (k vizinhos mais próximos)
17 k = 3
18 knn = KNeighborsClassifier(n_neighbors=k)
19
20 # 4. Treinando o modelo com os dados de treinamento
21 knn.fit(X_train, y_train)
22
23 # 5. Realizando previsões com os dados de teste
24 y_pred = knn.predict(X_test)
25
26 # 6. Avaliando o desempenho do modelo
27 accuracy = accuracy_score(y_test, y_pred)
28 print(f"Precisão do classificador k-MN (k={k}): {accuracy:.2f}")
29 print("\nRelatório de Classificação:")
30 print(classification_report(y_test, y_pred, target_names=iris.target_names))
31
32 # 7. Visualizando os dados em 2D (usando apenas 2 características para simplificar)
33 plt.figure(figsize=(8, 6))
34 cores = ['red', 'green', 'blue']
35 labels = iris.target_names
36
37 # Usamos apenas as duas primeiras características para plotar (comprimento e largura da sépala)
38 X_vis = X[:, :2]
39 X_train_vis, X_test_vis = train_test_split(X_vis, test_size=0.2, random_state=42)
40
41 # Reajustar o classificador para visualização
42 knn_vis = KNeighborsClassifier(n_neighbors=k)
43 knn_vis.fit(X_train_vis, y_train)
44
45 # Plotar pontos de teste
46 for i, cor in enumerate(cores):
47     plt.scatter(X_vis[y == i, 0], X_vis[y == i, 1], color=cor, label=labels[i], edgecolor='k')
48
49 plt.title(f'Classificação com k-MN (k={k}) - Visualização em 2D')
50 plt.xlabel('Comprimento da Sépala (cm)')
51 plt.ylabel('Largura da Sépala (cm)')
52 plt.legend()
53 plt.grid(True)
54 plt.show()
55

```

Figura 21

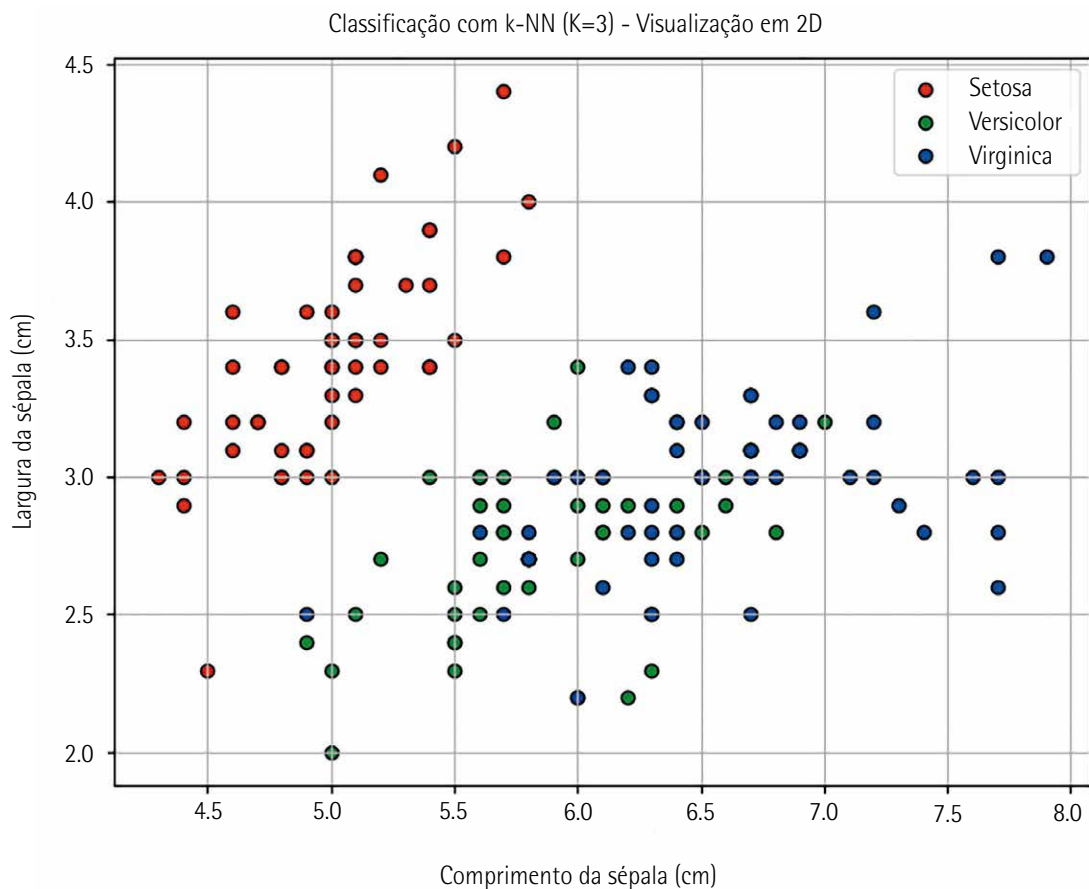


Figura 22

O algoritmo k-NN apresenta diversas vantagens que o tornam uma escolha popular em problemas de classificação e regressão. Ele é simples de entender e implementar, o que o torna acessível mesmo para iniciantes. Além disso, costuma apresentar bom desempenho em diferentes tipos de dados, especialmente quando as classes são bem definidas no espaço de atributos. Uma característica marcante do k-NN é que ele dispensa um processo de treinamento explícito, pois armazena os dados e toma decisões no momento da previsão, com base na comparação com os vizinhos mais próximos.



Observação

A eficácia do k-NN pode ser influenciada pelo valor de k escolhido e pela natureza do conjunto de dados. Experimente diferentes valores de k e verifique como isso afeta a precisão do classificador.

Por outro lado, o k-NN também possui limitações importantes. Em conjuntos de dados muito grandes, pode se tornar computacionalmente lento, já que precisa comparar uma nova entrada com todos os exemplos armazenados. Outro ponto sensível é que o algoritmo depende fortemente da escala dos dados – variáveis com magnitudes diferentes podem distorcer a noção de distância –, e também é vulnerável à presença de outliers, que podem influenciar negativamente os resultados.

Por fim, a escolha adequada do valor de k (número de vizinhos) é crucial: valores muito baixos podem gerar classificações instáveis, enquanto valores muito altos podem suavizar demais as decisões, prejudicando a precisão do modelo.

Quadro 8 – Comparativo entre os três algoritmos

Algoritmo	Tipo principal	Treinamento	Interpretação	Uso comum
Regressão linear	Regressão	Rápido	Alta	Previsão de valores contínuos
Regressão logística	Classificação binária	Rápido	Média	Diagnósticos, spam, previsões
k-NN	Classificação e regressão	Lento (na predição)	Baixa	Reconhecimento de padrões

Regressão linear, regressão logística e k-NN são, portanto, algoritmos fundamentais do aprendizado supervisionado. Cada um tem características específicas e é mais indicado para determinados tipos de problema. Compreender suas bases matemáticas, funcionamento e limitações é essencial para aplicá-los, de maneira eficaz, em soluções que envolvam previsões numéricas ou classificações automáticas. O domínio desses algoritmos é uma competência-chave para profissionais da área de inteligência artificial e ciência de dados.

3.3 Aplicações de classificação e regressão em IA

Os algoritmos de classificação e regressão são utilizados em sistemas de inteligência artificial aplicados a diferentes setores da sociedade. Com base em dados rotulados, essas técnicas supervisionadas permitem tomadas de decisão automáticas, previsões confiáveis e detecções inteligentes.

3.3.1 Aplicações de classificação

A classificação é usada sempre que o objetivo é atribuir uma categoria ou rótulo a uma entrada. Ela está presente em inúmeras soluções modernas baseadas em IA.

Saúde

Na área da saúde, especialmente em aplicações que envolvem classificação automatizada de dados clínicos, o objetivo é atribuir rótulos a novos dados com base em exemplos previamente rotulados. Essa técnica tem se mostrado extremamente eficaz em diversas aplicações clínicas. Um exemplo importante é o diagnóstico automático de doenças com base em sintomas ou resultados de exames.

Para isso, modelos supervisionados são treinados com conjuntos de dados que contêm registros médicos e históricos, onde os sintomas e exames dos pacientes estão associados a diagnósticos previamente confirmados. Após esse treinamento, o sistema é capaz de prever, com boa precisão, o provável diagnóstico de novos pacientes, auxiliando médicos na tomada de decisões mais rápidas e assertivas.

Outro exemplo relevante é a classificação de imagens médicas, como radiografias, tomografias ou mamografias. Utilizando algoritmos de aprendizado supervisionado, como redes neurais profundas, os modelos aprendem a identificar padrões visuais característicos de determinadas patologias, como a presença de tumores malignos em mamografias. Durante o treinamento, essas imagens são rotuladas por especialistas ("com câncer" ou "sem câncer", por exemplo), permitindo que o modelo aprenda a distinguir entre classes com base em características sutis das imagens. Uma vez treinado, o sistema pode analisar novas imagens e classificá-las automaticamente, reduzindo o tempo de análise e aumentando a precisão diagnóstica.

Além disso, a IA também tem sido empregada na triagem de pacientes em sistemas de pronto atendimento, utilizando algoritmos de classificação para atribuir níveis de prioridade (como grave, moderado ou leve) com base em variáveis como sintomas relatados, sinais vitais e histórico médico. Esses modelos são treinados com dados históricos rotulados por profissionais de saúde e, posteriormente, aplicados em tempo real para apoiar decisões críticas sobre a ordem de atendimento dos pacientes.

Essas aplicações demonstram como o aprendizado supervisionado, por meio de técnicas de classificação, tem potencial para transformar os serviços de saúde, tornando-os mais rápidos, eficientes e precisos na análise de dados clínicos complexos.

Segurança e detecção de fraudes

Um exemplo expressivo é a identificação de transações suspeitas em bancos e fintechs. Nesse caso, algoritmos supervisionados são treinados com históricos de transações financeiras previamente classificadas como legítimas ou fraudulentas. O modelo aprende, assim, a identificar características típicas de fraudes, como valores fora do padrão, horários incomuns ou transações em locais atípicos. Uma vez treinado, passa a sinalizar automaticamente novas transações com potencial risco, aumentando a eficácia dos sistemas de prevenção a fraudes.

Outro uso importante ocorre no reconhecimento facial, empregado para autenticação de identidade em sistemas de acesso, segurança pública e controle de fronteiras. Utilizando aprendizado supervisionado, o sistema é alimentado com um conjunto de imagens de rostos associadas a identidades conhecidas. A partir desse conjunto, o modelo aprende a extrair e comparar características biométricas, como distâncias entre olhos, formato do queixo ou textura da pele. Posteriormente, consegue reconhecer ou validar identidades em tempo real com alto grau de confiabilidade.

Na área da comunicação digital, a classificação de e-mails como spam ou phishing também é um exemplo clássico de aplicação de aprendizado supervisionado. Os algoritmos são treinados com milhares de mensagens previamente rotuladas como "spam", "phishing" ou "legítimas". Com base em padrões linguísticos, remetentes, estrutura do conteúdo e links suspeitos, o sistema aprende a distinguir os diferentes tipos de e-mail e aplica essa classificação automaticamente na caixa de entrada dos usuários, reduzindo riscos de fraudes e melhorando a experiência de uso.

Esses exemplos evidenciam como o aprendizado supervisionado, por meio da tarefa de classificação, é uma ferramenta poderosa para automatizar processos, aumentar a segurança e oferecer respostas rápidas e precisas em contextos críticos e cotidianos.

Indústria e manufatura

Na área da indústria e manufatura, um exemplo prático é a detecção de defeitos em produtos por meio da visão computacional. Imagens de peças e produtos são capturadas por câmeras e analisadas por algoritmos que foram previamente treinados com um conjunto de imagens rotuladas como "defeituosas" ou "em conformidade". Com isso, o sistema é capaz de identificar automaticamente rachaduras, deformações, imperfeições ou falhas sutis que comprometam a qualidade do item, garantindo um controle de qualidade mais preciso, rápido e com menor necessidade de intervenção humana.

Outro uso comum é a classificação de peças em esteiras automatizadas. Em ambientes industriais, onde diferentes tipos de componentes passam por esteiras de produção, sistemas de IA treinados por aprendizado supervisionado conseguem reconhecer e classificar cada peça com base em atributos, como forma, cor, tamanho ou código de identificação. Isso permite que o sistema direcione automaticamente os itens para os setores apropriados, otimizando o fluxo logístico e reduzindo erros de separação.

Além disso, o monitoramento do estado de equipamentos por meio de sistemas de manutenção preditiva também utiliza aprendizado supervisionado para classificar o funcionamento das máquinas. Os algoritmos analisam dados coletados por sensores, como temperatura, vibração, pressão e ruído, que foram previamente associados a estados normais ou a diferentes níveis de falha. Assim, o modelo pode prever quando um equipamento está operando normalmente ou quando há sinais de falha iminente, possibilitando ações corretivas antes que ocorram paradas inesperadas, reduzindo custos e aumentando a disponibilidade operacional.

Essas aplicações mostram como o uso do aprendizado supervisionado em tarefas de classificação vem transformando o setor industrial, tornando os processos mais inteligentes, autônomos e orientados por dados.

Varejo e marketing

No varejo e no marketing digital, o uso da inteligência artificial tem revolucionado a maneira como empresas compreendem e interagem com seus consumidores. A principal aplicação está na classificação de dados, em que algoritmos são treinados com conjuntos de exemplos rotulados para aprender padrões e realizar previsões em tempo real com alto grau de acerto.

Uma das aplicações mais estratégicas é a segmentação de clientes com base em seu comportamento de compra. Ao analisar dados históricos de transações, frequência de compras, ticket médio, preferências e padrões de navegação, modelos supervisionados aprendem a agrupar os consumidores em perfis, como compradores frequentes, esporádicos, sensíveis a promoções ou propensos à fidelização. Essa segmentação orientada por IA permite a personalização de campanhas de marketing, aumento da taxa de conversão e melhora da experiência do cliente.

Outro exemplo importante é a classificação automática de produtos por categorias em plataformas de e-commerce. A partir de descrições, imagens e características técnicas dos itens, algoritmos supervisionados treinados com exemplos rotulados conseguem identificar e classificar novos produtos

em categorias corretas, como "eletrônicos", "vestuário", "acessórios esportivos", entre outras. Essa automação reduz o trabalho manual, melhora a organização do catálogo digital e otimiza o sistema de busca dos usuários.

Além disso, a análise de sentimentos em comentários e avaliações de consumidores é outra aplicação relevante. Por meio de aprendizado supervisionado, modelos de IA são treinados com avaliações já rotuladas como positivas, negativas ou neutras, aprendendo a identificar padrões linguísticos, expressões emocionais e contextos que indicam a opinião do cliente. Com isso, o sistema é capaz de classificar automaticamente novos comentários, fornecendo às empresas insights valiosos sobre a percepção de seus produtos e serviços, e permitindo respostas ágeis a críticas ou elogios.

Essas aplicações demonstram como o aprendizado supervisionado, por meio de tarefas de classificação, tem sido essencial para tornar o varejo mais inteligente, proativo e centrado no cliente, promovendo decisões estratégicas baseadas em dados em tempo real.

Educação

Na área da educação, a aplicação da IA tem contribuído significativamente para a personalização do ensino, a gestão acadêmica e a melhoria do desempenho estudantil. Uma das aplicações mais relevantes é a classificação de alunos com risco de evasão.

Ao utilizar dados como frequência às aulas, desempenho em avaliações, participação em atividades virtuais, tempo de acesso às plataformas de ensino e histórico acadêmico, modelos supervisionados podem aprender a identificar perfis de estudantes que apresentam sinais de desengajamento ou dificuldades recorrentes. A partir dessa análise, o sistema classifica os alunos em níveis de risco (baixo, médio ou alto), possibilitando intervenções pedagógicas e administrativas mais rápidas e direcionadas para a retenção escolar.

Outra aplicação importante é a correção automática de provas objetivas, uma tarefa clássica de classificação. Os algoritmos são treinados com os gabaritos oficiais e aprendem a associar cada resposta dos alunos à alternativa correta ou incorreta. Essa automação permite uma correção rápida, precisa e em larga escala, otimizando o tempo dos professores e garantindo maior consistência na avaliação.

Além disso, a recomendação de trilhas de estudo com base no desempenho individual também se beneficia do aprendizado supervisionado. A partir do histórico de acertos e erros dos alunos em atividades anteriores, bem como de suas áreas de maior ou menor domínio, os modelos classificam o perfil de aprendizado do estudante e sugerem conteúdos personalizados, ajustando a trilha de estudo às suas necessidades específicas. Essa personalização favorece o aprendizado adaptativo e melhora o engajamento e os resultados acadêmicos.

Esses exemplos evidenciam como o uso de classificação supervisionada na educação tem potencial para transformar o acompanhamento pedagógico, tornando-o mais inteligente, preditivo e centrado nas necessidades reais de cada estudante.

3.3.2 Aplicações de regressão

A regressão é utilizada quando se deseja prever valores numéricos contínuos, permitindo projeções, simulações e estimativas com base em dados históricos.

Finanças

No setor financeiro, uma aplicação clássica é a previsão de preços de ações ou criptomoedas. Utilizando algoritmos de regressão treinados com dados históricos de preços, volume de transações, indicadores econômicos e até notícias de mercado, a IA é capaz de identificar padrões e tendências que auxiliam na estimativa de valores futuros. Esses modelos não apenas ajudam investidores a tomar decisões mais informadas, mas também são utilizados por sistemas automatizados de negociação (trading algorítmico).

Outra aplicação importante está na estimativa do risco de inadimplência com base no perfil de crédito do cliente. A partir de informações como renda, histórico de pagamento, tempo de vínculo empregatício, número de empréstimos anteriores e score de crédito, modelos de regressão supervisionada são treinados para prever a probabilidade de um cliente deixar de cumprir suas obrigações financeiras. Embora essa tarefa também possa envolver classificação (inadimplente ou não), o uso da regressão permite estimar, por exemplo, o valor esperado de perda ou o grau de risco em uma escala contínua, oferecendo uma análise mais refinada para instituições financeiras.

Além disso, os modelos de precificação dinâmica, como os aplicados à venda de passagens aéreas, hospedagens ou aluguel de veículos, também se beneficiam da regressão supervisionada. Esses modelos aprendem a prever os preços ideais com base em variáveis, como demanda, sazonalidade, tempo até a data do serviço, comportamento do consumidor e concorrência. Com isso, as empresas conseguem ajustar os preços em tempo real para maximizar a ocupação ou a margem de lucro.

Essas aplicações mostram como o aprendizado supervisionado com regressão é uma ferramenta poderosa para previsão e tomada de decisão no contexto financeiro, proporcionando mais precisão, eficiência e capacidade de adaptação a cenários complexos e voláteis.

Imobiliário

No setor imobiliário, a IA, por meio do aprendizado supervisionado com algoritmos de regressão, tem sido utilizada para realizar previsões de valores contínuos, especialmente em tarefas relacionadas à estimativa de preços de imóveis. Uma das aplicações mais comuns é a estimativa do valor de um imóvel com base em atributos como localização, metragem, número de quartos e banheiros, vagas de garagem, idade do imóvel, padrão de acabamento, entre outros.

Algoritmos de regressão são treinados com grandes bases de dados de imóveis previamente avaliados, permitindo que o modelo aprenda a identificar como cada característica influencia no preço final. Uma vez treinado, o sistema pode prever com boa precisão o valor de mercado de novos imóveis, auxiliando tanto corretores quanto compradores e investidores em suas decisões.

Outro uso relevante é na avaliação do impacto de reformas ou da localização no preço de venda de um imóvel. Por exemplo, a IA pode estimar quanto uma reforma na cozinha ou no banheiro pode valorizar o imóvel, ou ainda calcular o efeito de estar localizado próximo a centros comerciais, escolas ou estações de transporte público. Com base em dados históricos de imóveis antes e depois de reformas, ou localizados em diferentes regiões da cidade, os modelos de regressão conseguem quantificar esses impactos em valores monetários.

Essas aplicações demonstram como o uso de regressão supervisionada no setor imobiliário oferece uma abordagem analítica e orientada por dados para estimar preços e tomar decisões mais assertivas. Além de reduzir a subjetividade nas avaliações, essa tecnologia aumenta a transparência e eficiência no processo de compra, venda e investimento em imóveis.

Vendas e marketing

No campo de vendas e marketing, uma aplicação fundamental é a previsão de vendas com base em fatores como sazonalidade, ações promocionais, comportamento do consumidor e até mesmo condições climáticas. Utilizando algoritmos de regressão treinados com dados de vendas anteriores, esses sistemas conseguem identificar padrões e tendências, gerando previsões precisas de volume de vendas futuras. Isso permite um planejamento mais eficiente de estoques, logística e estratégias comerciais, reduzindo perdas e otimizando recursos.

Outra aplicação importante é o cálculo do Customer Lifetime Value (CLV), isto é, o valor que um cliente tende a gerar ao longo de seu relacionamento com a empresa. Com base em dados como frequência de compra, ticket médio, tempo de retenção e comportamento de consumo, modelos de regressão supervisionada estimam o valor monetário total que um cliente representará. Essa informação é essencial para definir orçamentos de marketing, estratégias de fidelização e decisões sobre aquisição de novos clientes.

Além disso, a análise do retorno sobre investimento de campanhas publicitárias também se beneficia da regressão. Ao cruzar dados de investimento em mídia, tempo de veiculação, segmentação e resultados obtidos (cliques, conversões ou vendas), os modelos conseguem prever o retorno financeiro esperado de campanhas futuras com características semelhantes. Isso ajuda as empresas a alocar melhor seus recursos e a avaliar com mais precisão a eficácia de cada ação de marketing.

Esses exemplos demonstram como os modelos de regressão supervisionada são ferramentas poderosas para transformar dados em insights acionáveis, permitindo que as áreas de vendas e marketing adotem uma postura mais preditiva, eficiente e orientada por resultados concretos.

Engenharia e manufatura

Na engenharia e na manufatura, uma das aplicações mais comuns é a estimativa do tempo de produção em linhas de montagem. Com base em dados, como tipo de produto, complexidade do processo, número de operadores disponíveis, eficiência dos equipamentos e histórico de produção, modelos de regressão são treinados para prever quanto tempo será necessário para concluir determinada etapa ou lote de fabricação. Essa previsão permite um gerenciamento melhor da capacidade produtiva, evita gargalos e melhora a eficiência geral da planta.

Outra aplicação relevante está na previsão do consumo de energia ou combustível em equipamentos industriais ou veículos. Os modelos de regressão supervisionada analisam variáveis como carga de trabalho, duração de operação, temperatura ambiente, tipo de equipamento e padrões de uso para estimar com precisão o consumo energético. Essa estimativa permite à empresa tomar decisões mais sustentáveis, reduzir custos operacionais e programar manutenções de maneira mais eficiente.

Além disso, a IA também é utilizada para simular o desgaste de componentes mecânicos com base em fatores como tempo de uso, intensidade da operação, ambiente (umidade, poeira, temperatura), e tipo de material. Os modelos de regressão supervisionada são treinados com dados históricos de falhas e durabilidade de peças semelhantes, o que possibilita prever quando determinado componente pode atingir um nível crítico de desgaste. Isso é fundamental para práticas de manutenção preditiva, que evitam falhas inesperadas e aumentam a vida útil dos equipamentos.

Essas aplicações demonstram como a regressão supervisionada tem papel estratégico na engenharia e manufatura, contribuindo para a automação inteligente, a economia de recursos e a melhoria contínua dos processos industriais. Ao transformar dados em previsões confiáveis, a IA permite uma produção mais eficiente, segura e orientada por métricas.

Saúde e ciências

Na área da saúde e nas ciências em geral, um exemplo importante é a previsão da evolução de doenças crônicas, como diabetes, hipertensão ou doenças cardíacas. A partir do histórico médico do paciente, incluindo exames laboratoriais, hábitos de vida, medicamentos utilizados e dados fisiológicos, como pressão arterial e glicemia, os modelos de regressão são treinados para estimar a progressão da condição ao longo do tempo. Isso permite uma atuação médica mais proativa, com ajustes personalizados no tratamento e maior prevenção de agravamentos.

Outro uso relevante é a estimativa de indicadores de crescimento físico em crianças, como peso, altura ou índice de massa corporal (IMC), com base em variáveis como idade, dieta, nível de atividade física e histórico familiar. Os modelos supervisionados aprendem com dados coletados de populações pediátricas e conseguem prever, de forma precisa, valores futuros esperados, auxiliando pediatras e nutricionistas na detecção precoce de desvios no desenvolvimento.

A modelagem do crescimento populacional ou da disseminação de vírus também se beneficia fortemente da regressão supervisionada. Com base em séries temporais e dados como taxa de natalidade, mobilidade urbana, densidade demográfica, clima e cobertura vacinal, os modelos conseguem projetar cenários futuros de expansão populacional ou de propagação de agentes infecciosos. Essas previsões são fundamentais para planejamento de políticas públicas, gestão hospitalar e estratégias de prevenção em larga escala.

Essas aplicações evidenciam como a regressão supervisionada amplia a capacidade analítica de profissionais da saúde e pesquisadores, ao oferecer previsões quantitativas, robustas e baseadas em dados reais. A IA, nesse contexto, atua como suporte decisório, promovendo ações mais precisas, personalizadas e eficientes em benefício da saúde individual e coletiva.

3.3.3 Integração das duas abordagens

A integração entre classificação e regressão, ambas técnicas do aprendizado supervisionado, é cada vez mais comum em sistemas inteligentes, pois tais abordagens podem atuar de modo complementar para oferecer análises mais completas e precisas. Enquanto a classificação lida com a categorização de dados em classes distintas, como "normal", "alerta" ou "crítico", a regressão permite prever valores contínuos, como tempo, temperatura ou indicadores fisiológicos.

Um exemplo prático dessa integração ocorre em sistemas de monitoramento industrial, nos quais a IA pode, simultaneamente, classificar o estado operacional de uma máquina, como "normal", "em alerta" ou "em estado crítico", e, ao mesmo tempo, prever o tempo estimado até a próxima falha, com base em dados de sensores, histórico de uso e condições de operação. Essa combinação permite não apenas identificar rapidamente situações de risco, mas também planejar ações preventivas com base em estimativas quantitativas.

Outro exemplo aparece em aplicativos de saúde personalizados. Um aplicativo pode classificar o risco cardíaco de um usuário como alto, médio ou baixo, a partir de variáveis como idade, histórico médico e hábitos de vida, e também prever a pressão arterial média esperada ao longo do dia, com base em padrões de alimentação, sono e atividade física. Ao unir essas duas abordagens, o sistema oferece uma visão mais abrangente, capaz de alertar para riscos iminentes e, ao mesmo tempo, orientar o usuário com dados preditivos para a adoção de comportamentos saudáveis.

A combinação de classificação e regressão, no mesmo sistema, promove análises mais completas, permitindo decisões mais informadas, personalizadas e eficazes em diversos contextos – da indústria à saúde.

Impacto nas decisões e na automação

As técnicas de classificação e regressão não se limitam à geração de insights; elas são fundamentais na automatização de decisões em tempo real. Ao permitir que sistemas analisem grandes volumes de dados e façam previsões ou categorizações precisas, essas abordagens tornam-se essenciais para a construção de soluções inteligentes e autônomas.

Em aplicações práticas, os modelos de classificação podem, por exemplo, ativar alertas automáticos quando identificam padrões associados a comportamentos de risco ou anomalias operacionais. Do mesmo modo, podem direcionar o atendimento humano para os casos mais críticos, com base na priorização automática de chamados, pacientes ou processos. Já os modelos de regressão permitem, por exemplo, ajustar preços dinamicamente em e-commerces, serviços de transporte ou hospedagem, com base na previsão de demanda, estoque ou variáveis externas como clima e horário.

Tanto a classificação quanto a regressão podem acionar ações corretivas automaticamente, como a desaceleração de uma máquina prestes a falhar, o envio de notificações personalizadas a um usuário ou a alteração de parâmetros de operação em sistemas industriais. Essa capacidade de agir, com base em dados em tempo real, é o que torna essas técnicas tão poderosas: elas não apenas interpretam o

passado e projetam o futuro, mas também transformam diretamente o presente por meio de decisões automatizadas e inteligentes.

Assim, o impacto do aprendizado supervisionado vai muito além da análise. Ele redefine a maneira como os sistemas modernos tomam decisões, otimizam recursos e respondem de forma proativa aos desafios do ambiente em que operam.

3.4 Redes neurais

As redes neurais artificiais (RNAs) são modelos computacionais inspirados no funcionamento do cérebro humano. Elas formam a base de várias técnicas avançadas de IA, especialmente em áreas como reconhecimento de imagem, linguagem natural e previsão de padrões complexos. Sua capacidade de aprender com dados e generalizar soluções tornou as redes neurais importantes no avanço da IA moderna.

As RNAs são estruturas compostas por unidades chamadas neurônios artificiais, organizadas em camadas, que processam informações por meio de conexões ajustáveis (pesos). Cada neurônio recebe entradas, aplica um cálculo e gera uma saída que alimenta o próximo neurônio da rede.

O neurônio de McCulloch e Pitts (figura 23), proposto em 1943 pelos pesquisadores Warren McCulloch e Walter Pitts, é considerado o modelo teórico pioneiro das redes neurais artificiais. Inspirado no funcionamento dos neurônios biológicos, esse modelo representa um neurônio artificial simplificado capaz de realizar operações lógicas básicas.

Ele recebe múltiplas entradas binárias (0 ou 1), cada uma associada a um peso, realiza uma soma ponderada dessas entradas e, com base em um limiar (threshold), emite uma saída também binária – ativando-se (saída 1) apenas quando a soma atinge ou ultrapassa esse valor.

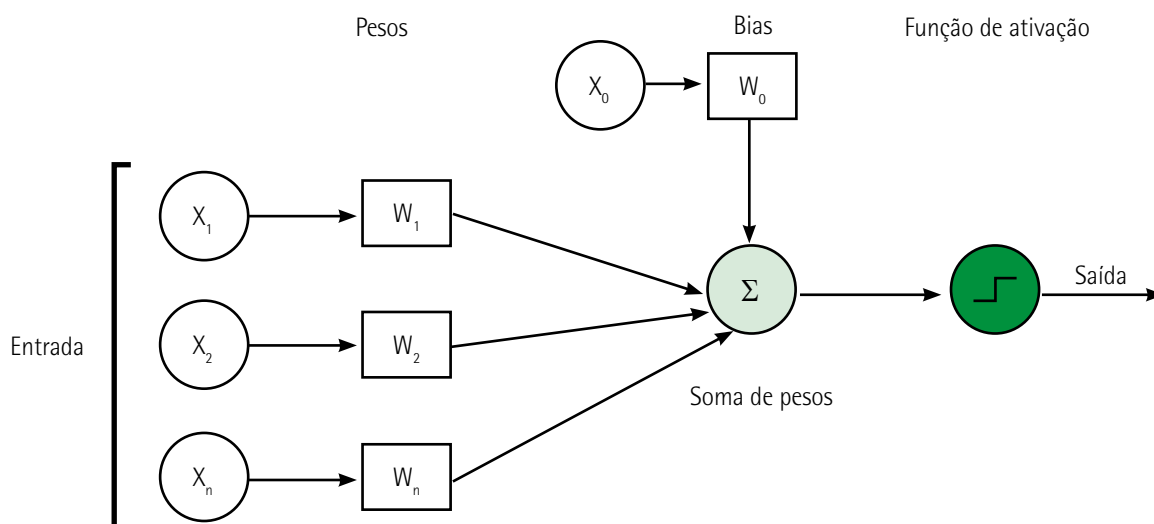


Figura 23 – Neurônio de McCulloch e Pitts

Adaptada de: Catarino (2025, p. 56).

Apesar de sua estrutura simples e de não incorporar conceitos como aprendizado ou ajustes de pesos, o modelo de McCulloch e Pitts foi fundamental para demonstrar que redes de neurônios artificiais poderiam ser utilizadas para simular funções computacionais e raciocínio lógico, lançando as bases para o desenvolvimento das redes neurais modernas.

3.4.1 Arquitetura básica de uma rede neural

A estrutura de uma rede neural é dividida em três partes principais.

- **Camada de entrada (input layer):** recebe os dados de entrada.
- **Camadas ocultas (hidden layers):** realizam o processamento interno e aprendizado.
- **Camada de saída (output layer):** fornece o resultado final da rede.

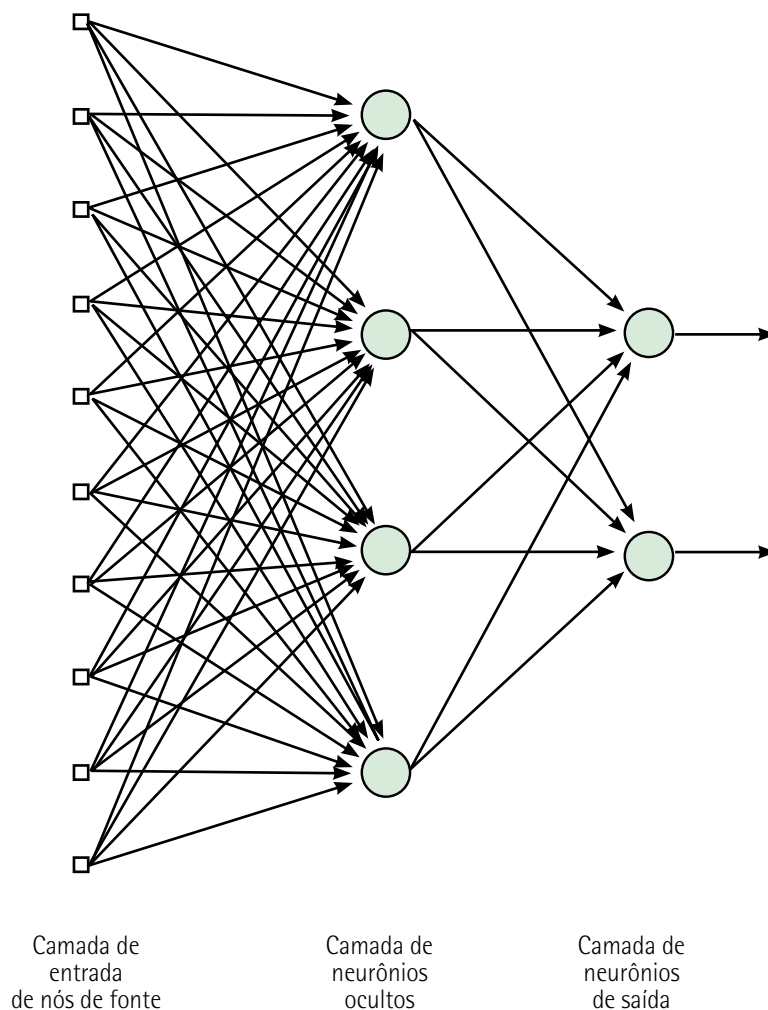


Figura 24 – Exemplo de rede neural

Adaptada de: Haykin (2007, p. 48).

Cada neurônio calcula uma função de ativação, que define se o sinal será ativado e passado adiante.

Exemplo de funções de ativação:

- **ReLU (unidade linear retificada):** usada em redes profundas.
- **Sigmoide:** usada em classificação binária.
- **Softmax:** usada para classificação multiclasse.

Em um neurônio artificial, a função de ativação é uma função matemática que é aplicada à soma ponderada dos sinais de entrada para determinar a saída do neurônio. Essa função é responsável por determinar se o neurônio deve ou não ser ativado e, se sim, qual deve ser a intensidade da ativação.

Existem diversas funções de ativação que podem ser usadas em um neurônio artificial, e cada uma delas apresenta características específicas. As mais comuns são:

- **Função degrau (step function):** a saída do neurônio é 1, se a soma ponderada dos sinais de entrada for maior que um determinado limiar, e 0, caso contrário. Essa função é simples e eficiente, mas não é diferenciável e pode apresentar problemas durante o treinamento de redes neurais.
- **Função sigmoideal:** essa função tem formato de S e é diferenciável em todos os pontos. Ela é usada principalmente em problemas de classificação binária, em que a saída do neurônio deve ser um valor entre 0 e 1. A função sigmoideal também é usada em redes neurais profundas, onde ajuda a reduzir a variância e melhorar a estabilidade do treinamento.
- **Função tangente hiperbólica:** similar à função sigmoideal, mas com valores de saída entre -1 e 1. Ela é amplamente utilizada em redes neurais devido à sua capacidade de modelar relações não lineares e sua propriedade de simetria em torno do eixo y.
- **Função ReLU (rectified linear unit):** essa função é definida como $f(x) = \max(0, x)$. Ela é utilizada em redes neurais profundas devido à sua eficiência computacional e à sua capacidade de lidar com o problema do gradiente desvanecente.
- **Função softmax:** é usada em problemas de classificação multiclasse, onde a saída do neurônio deve ser uma distribuição de probabilidade sobre as possíveis classes. Ela converte as saídas dos neurônios em valores de probabilidade normalizados, garantindo que a soma de todas as saídas seja igual a 1.

A escolha da função de ativação depende do problema em questão e das características da rede neural. Cada função tem suas vantagens e desvantagens, e o uso de uma função inadequada pode levar a problemas de desempenho e convergência durante o treinamento.



Saiba mais

Sobre função de ativação, leia o seguinte artigo:

NWANKPA, C. *et al.* Activation functions: comparison of trends in practice and research for deep learning. *Arxiv* (Cornell University), 8 nov. 2018. Disponível em: <https://x.gd/C8iqT>. Acesso em: 25 jul. 2025.

3.4.2 Funcionamento de uma rede neural

O processo de aprendizado em uma rede neural acontece em duas etapas principais.

- **Propagação direta (forward propagation):** os dados de entrada passam pelas camadas da rede, sendo processados por cada neurônio até gerar uma saída final.
- **Retropropagação do erro (backpropagation):** a saída da rede é comparada com a saída esperada. Com base no erro, os pesos das conexões são ajustados utilizando algoritmos de otimização, como o gradiente descendente. Esse processo se repete várias vezes durante o treinamento.

Os tipos de redes neurais são:

- **Perceptron simples:** é o modelo mais básico de rede neural, composto por uma única camada de neurônios. Ele é capaz de resolver apenas problemas linearmente separáveis, ou seja, aqueles em que as classes podem ser divididas por uma linha reta (ou um hiperplano, em dimensões maiores). Apesar de suas limitações, o perceptron simples é importante na evolução das redes neurais.
- **Perceptron de múltiplas camadas (multilayer perceptron – MLP):** esse modelo expande a estrutura do perceptron simples ao incluir uma ou mais camadas ocultas entre a entrada e a saída. Com essa arquitetura, o MLP é capaz de aprender relações não lineares complexas nos dados, tornando-o uma ferramenta versátil tanto para tarefas de classificação quanto de regressão. É uma das redes mais utilizadas em aplicações gerais de aprendizado supervisionado.
- **Redes neurais convolucionais (convolutional neural networks – CNNs):** projetadas especialmente para o reconhecimento e processamento de imagens, as CNNs utilizam filtros (ou kernels) que percorrem a imagem para identificar padrões visuais, como bordas, formas e texturas. Graças à sua capacidade de extrair características espaciais com eficiência, essas redes são muito utilizadas em visão computacional, diagnóstico por imagem e reconhecimento facial.
- **Redes neurais recorrentes (recurrent neural networks – RNNs):** são redes indicadas para o processamento de dados sequenciais, como séries temporais, texto, áudio e linguagem natural. A principal característica das RNNs é a presença de conexões recorrentes, que permitem que a rede "lembre" de informações anteriores em uma sequência, o que é essencial para modelar dependências temporais ou contextuais nos dados.

3.4.3 Aplicações práticas de redes neurais

As redes neurais artificiais têm se destacado como uma das ferramentas mais poderosas da inteligência artificial, sendo utilizadas em diversas áreas para resolver problemas complexos, aprender padrões em grandes volumes de dados e automatizar processos decisórios. Na área da saúde, redes neurais são aplicadas em sistemas de diagnóstico por imagem, como na detecção de tumores em mamografias ou lesões em exames de ressonância magnética, além de auxiliar na previsão de doenças crônicas com base em históricos médicos e exames laboratoriais.

No setor financeiro, redes neurais são utilizadas para a detecção de fraudes em transações bancárias, identificando padrões suspeitos em tempo real, e para a previsão de mercados financeiros, analisando tendências com base em séries temporais, indicadores econômicos e notícias. No varejo, essas redes alimentam sistemas de recomendação de produtos personalizados, com base no comportamento de compra dos consumidores, e ajudam na análise preditiva do perfil de clientes, otimizando campanhas de marketing e estratégias de fidelização.

Na área de transporte, as redes neurais são fundamentais para o desenvolvimento de veículos autônomos, processando imagens, sensores e dados de tráfego para tomar decisões de navegação em tempo real. Elas também são utilizadas na otimização de rotas logísticas, considerando variáveis como distância, tempo e custo. No campo da segurança, redes neurais profundas (deep learning) são aplicadas em sistemas de reconhecimento facial e biometria, garantindo maior precisão na identificação de indivíduos e no controle de acesso a ambientes restritos.

No setor educacional, as redes neurais contribuem para a correção automática de avaliações objetivas e discursivas, além de possibilitar a previsão do desempenho acadêmico de estudantes, permitindo intervenções pedagógicas mais eficazes. Já na área de processamento de linguagem natural (PLN), elas são empregadas em sistemas de tradução automática, chatbots inteligentes e assistentes virtuais, que compreendem e respondem a comandos de voz ou texto de maneira cada vez mais natural.

Essas aplicações demonstram como as redes neurais estão transformando setores diversos da sociedade, ao oferecerem soluções inteligentes, adaptáveis e altamente eficientes, contribuindo para avanços significativos na automação, na personalização de serviços e na tomada de decisões baseada em dados.

Vantagens e desvantagens

As redes neurais artificiais oferecem diversas vantagens que as tornam uma das abordagens mais eficazes no campo da IA. Uma de suas principais qualidades é a capacidade de modelar relações complexas entre variáveis, o que as torna ideais para problemas que envolvem padrões não lineares ou interdependências sutis nos dados. Além disso, são extremamente versáteis, podendo ser aplicadas com sucesso em uma ampla variedade de domínios, como saúde, finanças, transporte, marketing, entre outros.

Outro ponto forte é o aprendizado automático de características, especialmente em redes profundas, onde a própria rede é capaz de extrair e combinar atributos relevantes dos dados brutos sem a necessidade de intervenção manual. Essa capacidade resulta em alta precisão em tarefas específicas, como reconhecimento de imagens, detecção de fraudes ou previsão de séries temporais.

No entanto, o uso de redes neurais também apresenta algumas desvantagens importantes. Uma delas é a necessidade de grandes volumes de dados para o treinamento adequado – quanto mais complexa a rede, mais exemplos são necessários para que ela generalize bem. Essas redes são frequentemente criticadas por sua baixa interpretabilidade, funcionando como verdadeiras “caixas-pretas”, o que dificulta a compreensão dos critérios usados nas decisões ou previsões realizadas. O alto custo computacional, especialmente em arquiteturas profundas, também é um desafio, exigindo hardware especializado, como GPUs, e longos tempos de processamento.

Por fim, há o risco de overfitting, ou seja, de a rede se ajustar excessivamente aos dados de treinamento, perdendo a capacidade de generalizar para novos exemplos – um problema que pode ser mitigado com técnicas de regularização adequadas.

Assim, embora as redes neurais ofereçam resultados impressionantes, sua adoção exige cuidados técnicos e estruturais, principalmente relacionados à qualidade dos dados, à interpretabilidade e ao desempenho computacional.



Observação

Overfitting é um problema comum em redes neurais e ocorre quando a rede aprende padrões específicos dos dados de treinamento que não generalizam bem para novos dados de teste. Em outras palavras, a rede neural se ajusta muito bem aos dados de treinamento, mas não consegue generalizar para novos exemplos.

Esse problema, geralmente, ocorre quando a rede é muito complexa em relação ao tamanho do conjunto de dados de treinamento ou quando o conjunto de treinamento não é representativo o suficiente em relação aos dados que a rede neural encontrará na vida real. Como resultado, a rede aprende padrões que são específicos para o conjunto de treinamento e não são relevantes para outros dados.

O overfitting pode ser identificado observando o desempenho da rede neural nos dados de treinamento e de teste. Se o desempenho da rede for muito bom nos dados de treinamento, mas ruim nos dados de teste, é provável que a rede esteja sofrendo de overfitting.

Considerando tais aspectos, imagine uma rede neural treinada para identificar dígitos manuscritos (0 a 9). As imagens são convertidas em vetores de pixels (entrada), processadas por camadas ocultas que reconhecem padrões (curvas, ângulos, preenchimentos), e ao final a camada de saída retorna à probabilidade de cada número, sendo selecionado o de maior valor.

As redes neurais representam um dos pilares da IA atual, permitindo a criação de sistemas com alta capacidade de reconhecimento, previsão e adaptação. Seu entendimento é essencial para qualquer profissional que deseje atuar com IA aplicada a problemas reais, desde simples classificadores até sistemas avançados com aprendizado profundo.

3.4.4 Estrutura e funcionamento de redes neurais artificiais

As RNAs são modelos computacionais inspirados no funcionamento do cérebro humano, compostos por camadas de unidades de processamento (neurônios artificiais) que trabalham em conjunto para reconhecer padrões e resolver problemas complexos. Compreender a estrutura e o funcionamento interno das RNAs é essencial para entender como sistemas inteligentes aprendem a partir de dados.

Componentes de uma rede neural

Uma rede neural artificial é composta por uma arquitetura em camadas que imita, simplificada, o funcionamento dos neurônios biológicos. Essa estrutura é dividida em três tipos principais de camadas: camada de entrada, camadas ocultas e camada de saída. Cada uma possui um papel específico no processamento das informações.

- **Camada de entrada (input layer):** a camada de entrada é responsável por receber os dados brutos que serão processados pela rede. Nela, cada neurônio representa uma variável de entrada do problema. Não há processamento nessa etapa – ela apenas repassa as informações para as camadas seguintes. Por exemplo: em um sistema de reconhecimento de dígitos manuscritos com imagens de 28x28 pixels, cada pixel da imagem é transformado em um número (por exemplo, entre 0 e 255, representando intensidade de cor). A camada de entrada, nesse caso, terá 784 neurônios ($28 \times 28 = 784$), um para cada pixel, que repassam essas informações à próxima camada.

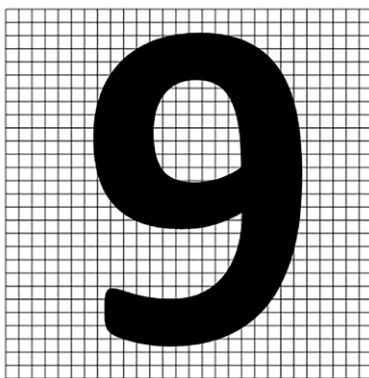


Figura 25 – Exemplo de uma imagem considerando 28x28 pixels

- **Camadas ocultas (hidden layers):** as camadas ocultas são o coração da rede neural. É nelas que ocorre o processamento e o aprendizado. Cada neurônio de uma camada oculta:
 - Recebe os dados da camada anterior, multiplicando cada entrada por um peso específico.
 - Soma os valores ponderados e adiciona um termo chamado viés (bias).
 - Aplica uma função de ativação (como ReLU, sigmoid ou tanh) para transformar essa soma em uma saída que será repassada à próxima camada.

Essas camadas são chamadas ocultas porque não interagem diretamente com os dados de entrada nem com a saída final – estão “escondidas” entre elas. A quantidade de camadas ocultas e o número de neurônios em cada uma determinam a capacidade da rede de aprender padrões complexos.

Redes com mais camadas (conhecidas como redes profundas) são capazes de aprender relações mais sofisticadas, mas também exigem mais dados e poder computacional. Por exemplo, em uma rede usada para identificar emoções faciais, as camadas ocultas podem aprender a detectar características como bordas dos olhos, expressões de boca e contornos do rosto, combinando essas informações para reconhecer emoções como “alegria” ou “tristeza”.

- **Camada de saída (output layer):** a camada de saída é a última etapa do processamento. Ela gera o resultado final da rede, que depende do tipo de problema a ser resolvido:
 - **Um único neurônio:** usado em regressão, quando se deseja prever um valor contínuo (como a temperatura de amanhã).
 - **Dois neurônios:** comum em classificação binária, como detectar se um e-mail é “spam” ou “não spam”.
 - **Vários neurônios:** usado em classificação multiclasse, como identificar qual número de 0 a 9 está presente em uma imagem. Nesse caso, cada neurônio representa uma classe possível e a saída indica a probabilidade de cada uma.

Consideremos como exemplo que em um aplicativo de reconhecimento de dígitos, a camada de saída terá 10 neurônios, representando os números de 0 a 9. A rede fornecerá como resultado a probabilidade de cada número ser o correto e o valor com maior probabilidade será o escolhido como resposta.

Esses três componentes – entrada, camadas ocultas e saída – formam a base das redes neurais. A forma como se conectam, o número de camadas e os parâmetros ajustados durante o treinamento definem o desempenho da rede na resolução de problemas reais.

Funcionamento de uma rede neural

O funcionamento de uma rede neural artificial pode ser compreendido em duas fases principais: propagação direta (forward propagation) e retropropagação do erro (backpropagation). Essas etapas

ocorrem durante o treinamento da rede e são fundamentais para que ela aprenda a fazer previsões precisas com base em dados.

- **Propagação direta (forward propagation):** na propagação direta, os dados percorrem a rede no sentido da entrada até a saída. É nesse momento que a rede processa as informações e gera uma previsão com base nos pesos e parâmetros atuais. Etapas da propagação direta:
 - Os neurônios da camada de entrada recebem os dados brutos (como pixels de uma imagem, números de um formulário ou sensores) e enviam esses valores para os neurônios da primeira camada oculta.
 - Cada neurônio da camada oculta recebe essas entradas, multiplica por pesos associados, soma os resultados, adiciona um viés (bias) e aplica uma função de ativação (como ReLU, sigmoid ou tanh) para gerar sua própria saída.
 - Esse processo se repete, camada por camada, até que os sinais alcancem a camada de saída.
 - A saída final da rede representa a previsão do modelo, que pode ser um número contínuo (em problemas de regressão) ou uma classe (em problemas de classificação).

Consideremos uma rede neural treinada para prever o preço de um imóvel com base em localização, metragem e número de quartos. Na propagação direta, essas variáveis de entrada percorrem a rede, passam por cálculos nas camadas ocultas e geram como saída o valor estimado de venda.

- **Retropropagação do erro (backpropagation):** após a rede gerar uma previsão, o próximo passo é verificar o quão distante essa previsão está do valor real (fornecido nos dados rotulados). Essa diferença é chamada de erro, e a retropropagação é o processo pelo qual esse erro é distribuído de volta pela rede, camada por camada, a fim de ajustar os pesos e melhorar a precisão do modelo. As etapas da retropropagação são:
 - A rede compara a saída gerada com a resposta correta (rótulo real) e calcula o erro.
 - Esse erro é propagado no sentido inverso da camada de saída até a camada de entrada, determinando quanto cada peso contribuiu para o erro total.
 - Com base nessa análise, os pesos das conexões são ajustados para que, na próxima vez que os mesmos dados forem apresentados, o erro seja menor.

Voltando ao exemplo da previsão de preço de imóveis, se a rede previu R\$ 350.000, mas o valor real era R\$ 400.000, a retropropagação ajustará os pesos para que, em futuras situações semelhantes, a rede aprenda a prever valores mais próximos da realidade.

Observação

Gradiente descendente é o algoritmo mais utilizado para fazer os ajustes. Ele calcula o gradiente (a inclinação) da função de erro em relação a cada peso, determinando a direção de maior redução do erro.

Taxa de aprendizado (learning rate) define o tamanho do passo dado na direção do gradiente. Se for muito alta, pode causar instabilidade; se for muito baixa, o treinamento será lento.

Essas duas fases – propagação direta e retropropagação – acontecem repetidamente durante o processo de treinamento, geralmente por milhares de iterações (ou épocas), até que a rede aprenda a fazer previsões com um nível de erro aceitável. Esse ciclo de aprendizado contínuo é o que torna as redes neurais tão eficazes em problemas complexos e variados.

Neurônio artificial

O neurônio artificial é a unidade básica de processamento. Ele realiza três operações principais:

- multiplica os valores de entrada pelos pesos correspondentes;
- soma os valores ponderados;
- aplica uma função de ativação ao resultado.

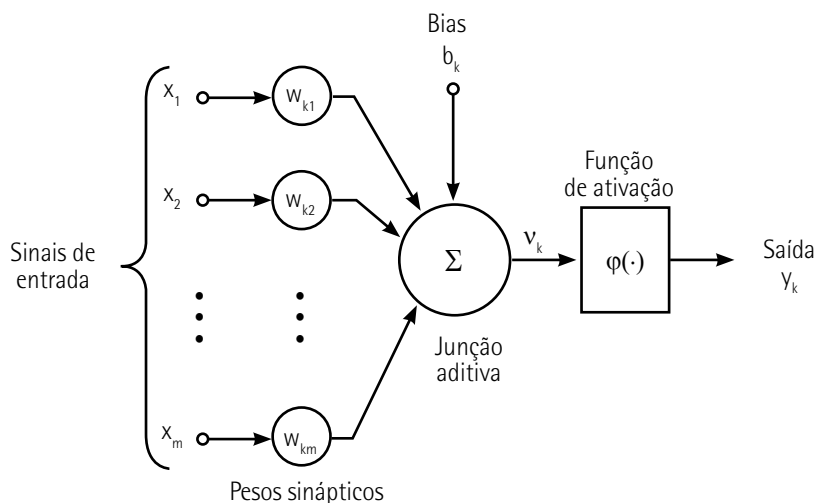


Figura 26 – Modelo de neurônio artificial

Adaptada de: Haykin (2007, p. 38).

Equação matemática:

$$\text{saída} = f(w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b)$$

Onde:

- x são as entradas;
- w são os pesos;
- b é o viés (bias);
- f é a função de ativação.

Funções de ativação

Controlam a saída dos neurônios.

Quadro 9 – Funções de ativação mais comuns

Função	Características	Aplicação comum
ReLU	$f(x) = \max(0, x)$ – simples e eficiente	Redes profundas (deep learning)
Sigmoide	$f(x) = 1 / (1 + e^{(-x)})$ – saída entre 0 e 1	Classificação binária
Tanh	Saída entre -1 e 1	Modelos onde polaridade é relevante
Softmax	Distribuição de probabilidades entre classes	Classificação multiclasse

A partir de tais informações, considere uma rede neural que recebe como entrada as notas de um aluno (provas, trabalhos e frequência) e deve prever se ele será "aprovado" ou "reprovado".

As notas, então, são inseridas como entrada e passam pela camada oculta, em que os valores são combinados com pesos e processados. Assim, a saída final indica a probabilidade de aprovação. Com base no resultado, a rede ajusta os pesos em caso de erro durante o treinamento.

Parâmetros importantes:

- **Pesos (weights):** determinam a influência de cada entrada.
- **Bias (viés):** permitem que o neurônio aprenda deslocamentos no espaço de decisão.
- **Épocas:** número de vezes que o modelo percorre o conjunto de treinamento completo.
- **Batch size:** quantidade de exemplos processados antes da atualização dos pesos.
- **Overfitting:** quando a rede aprende demais os dados de treino e perde capacidade de generalização.

A estrutura e o funcionamento das redes neurais artificiais são a base do aprendizado profundo (deep learning). Compostas por camadas interligadas de neurônios, essas redes aprendem a reconhecer padrões complexos a partir de exemplos. Dominar esses conceitos é fundamental para entender como sistemas de IA modernos operam, desde assistentes virtuais até reconhecimento de voz, imagem e tomada de decisão inteligente.

3.4.5 Algoritmos de retropropagação e aprendizado em redes neurais

O verdadeiro poder das redes neurais está em sua capacidade de aprender com dados, ajustando automaticamente os pesos das conexões entre neurônios com base nos erros cometidos durante o treinamento. Esse processo é viabilizado por um algoritmo fundamental: a retropropagação do erro (ou backpropagation). Desse modo, exploraremos agora como as redes neurais aprendem, o papel da retropropagação e os algoritmos de otimização envolvidos nesse processo.

O aprendizado de uma rede neural consiste em ajustar os pesos e os vieses de seus neurônios com o objetivo de minimizar o erro entre as saídas previstas pela rede e os valores reais esperados.

Esse processo ocorre de maneira iterativa e envolve:

- **Propagação direta (forward propagation):** os dados entram na rede e uma previsão é feita.
- **Cálculo do erro:** compara-se a saída prevista com a saída real.
- **Retropropagação do erro:** o erro é propagado de volta pela rede para ajustar os pesos.
- **Atualização dos pesos:** baseado no erro, os pesos são corrigidos com o objetivo de melhorar as próximas previsões.

Retropropagação (backpropagation)

A retropropagação do erro é um algoritmo de otimização baseado em cálculo de derivadas parciais, usado para atualizar os pesos da rede com base no gradiente do erro.

As derivadas parciais são um conceito fundamental no cálculo multivariado e descrevem como uma função com múltiplas variáveis muda em relação a uma única variável, mantendo as demais constantes. Por exemplo, se temos uma função $f(x,y)$ que depende de duas variáveis, a derivada parcial de f , em relação a x , indica a taxa de variação de f ao longo do eixo x , assumindo que y permaneça fixo.

Esse conceito é aplicado na otimização de funções multivariadas, como no treinamento de redes neurais, onde as derivadas parciais são utilizadas para calcular os gradientes necessários na atualização dos pesos durante o processo de aprendizado.

Primeiramente, calcula-se o erro na saída final da rede. Esse erro é distribuído "para trás" pelas camadas da rede. Assim, cada neurônio recebe sua "parcela de culpa" no erro total, e os pesos de cada conexão são ajustados de forma proporcional a essa culpa.

A seguir, temos a equação de atualização dos pesos (regra do gradiente descendente):

$$w \leftarrow w - \eta \cdot \partial E / \partial w$$

Onde:

- w é o peso atual da conexão;
- η (eta) é igual à taxa de aprendizado;
- $\partial E / \partial w$ é a derivada do erro em relação ao peso.

Essa fórmula diminui o erro passo a passo, ajustando os pesos para descer em direção ao mínimo da função de erro.



Saiba mais

Sobre derivadas parciais, leia o texto indicado a seguir.

TELLES, D. A.; YAMASHIRO, S.; SOUZA, S. A. O. Derivadas de funções de várias variáveis. In: TELLES, D. A.; YAMASHIRO, S.; SOUZA, S. A. O. *Matemática com aplicações tecnológicas: cálculo II*, volume 3. 2. ed. São Paulo: Blucher, 2020.

Função de erro (função de custo)

A função de erro mede o quão distante a saída prevista está da saída desejada. Algumas das mais utilizadas são:

Quadro 10

Função de erro	Aplicação comum
Erro quadrático médio (MSE)	Problemas de regressão
Entropia cruzada (cross-entropy)	Classificação binária e multiclasse

Taxa de aprendizado (learning rate)

A taxa de aprendizado (η) define o tamanho dos passos dados na direção de minimização do erro. A taxa muito alta pode causar instabilidade, e a taxa muito baixa pode gerar um aprendizado lento.

É comum usar técnicas como decaimento da taxa ou otimizadores adaptativos para ajustar esse parâmetro dinamicamente.

Otimizadores baseados em gradiente

Além do gradiente descendente tradicional, existem variações modernas que aceleram o treinamento e evitam armadilhas como mínimos locais.

No treinamento de redes neurais, os otimizadores são algoritmos fundamentais e responsáveis por ajustar os pesos da rede com o objetivo de minimizar a função de erro. Existem diversos otimizadores, cada um com características específicas que afetam o desempenho e a velocidade de convergência do modelo.

Um dos mais simples é o gradiente descendente estocástico (stochastic gradient descent – SGD), que atualiza os pesos após cada amostra de treino, tornando o processo mais rápido e eficiente em termos de memória. No entanto, por ser altamente sensível ao ruído dos dados, ele pode apresentar flutuações no caminho de descida, o que dificulta a convergência suave.

Para contornar esse problema, o momentum foi introduzido como uma melhoria ao SGD. Esse método considera a direção do gradiente nas iterações anteriores, criando um tipo de “inércia” que suaviza o caminho do aprendizado e ajuda a escapar de mínimos locais, acelerando o processo de otimização.

Outro otimizador importante é o RMSProp, que ajusta dinamicamente a taxa de aprendizado para cada parâmetro, dividindo o gradiente por uma média móvel dos quadrados dos gradientes passados. Isso permite que o modelo aprenda de modo mais estável, especialmente em funções com curvaturas diferentes em cada dimensão.

Por fim, o adaptive moment estimation (Adam) é atualmente um dos otimizadores mais utilizados. Ele combina as vantagens do momentum e do RMSProp, utilizando tanto a média dos gradientes quanto a média dos seus quadrados para adaptar as atualizações de maneira eficiente e robusta. O Adam se destaca por sua boa performance mesmo com pouca necessidade de ajustes manuais na taxa de aprendizado, sendo ideal para uma variedade de problemas e arquiteturas.

Agora podemos considerar as etapas do aprendizado supervisionado com retropropagação:

- **Inicialização:** pesos e vieses da rede são definidos com valores aleatórios.
- **Forward propagation:** dados entram e são processados até a saída.
- **Cálculo do erro:** compara a saída prevista com o rótulo verdadeiro.
- **Backpropagation:** o erro é propagado de volta para calcular os gradientes.
- **Atualização dos pesos:** os pesos são ajustados com base nos gradientes.
- **Repetição:** o processo é repetido por várias épocas, até que o erro seja minimizado.

Exemplos práticos de aprendizado com backpropagation são:

- **Reconhecimento de dígitos manuscritos:** a rede aprende com milhares de exemplos rotulados (exemplo: banco de dados MNIST).
- **Análise de sentimentos:** a rede recebe textos e aprende a classificá-los como positivos ou negativos.
- **Deteção de fraudes:** a rede aprende padrões de comportamento que indicam transações fraudulentas.

Por fim, há ainda os desafios e cuidados no treinamento. São eles:

- **Overfitting:** a rede aprende demais os dados de treinamento e falha com novos dados. Solução: regularização, dropout, aumento de dados.
- **Underfitting:** a rede é muito simples e não consegue aprender os padrões. Solução: usar uma arquitetura mais robusta.
- **Escolha da arquitetura:** número de camadas e neurônios pode afetar drasticamente o desempenho.

A retropropagação do erro é o algoritmo central para o treinamento de redes neurais. Com ela, os pesos das conexões são ajustados automaticamente com base nos erros cometidos, permitindo que a rede aprenda a resolver tarefas complexas, como classificação de imagens, tradução automática, diagnóstico médico e muito mais. Combinada com otimizadores modernos e grandes volumes de dados, a retropropagação tornou-se fundamental no avanço das aplicações de deep learning.

4 APRENDIZADO NÃO SUPERVISIONADO E REFORÇADO

Além do aprendizado supervisionado, em que os dados possuem rótulos conhecidos, a inteligência artificial também conta com outros dois paradigmas fundamentais de aprendizado: o aprendizado não supervisionado e o aprendizado por reforço. Cada abordagem é usada para resolver tipos distintos de problemas e contribui para a construção de sistemas mais inteligentes e adaptativos.

No aprendizado não supervisionado, os dados de entrada não possuem rótulos ou saídas associadas. O objetivo do algoritmo é descobrir estruturas ocultas, padrões ou agrupamentos dentro dos dados. A máquina aprende sozinha, identificando similaridades e diferenças entre os dados fornecidos. As principais tarefas são:

- **Agrupamento (clustering):** segmenta dados semelhantes em grupos.
- **Redução de dimensionalidade:** simplifica grandes conjuntos de dados mantendo suas características essenciais.

- **Deteção de anomalias:** identifica dados que se desviam de padrões normais.
- **Associação:** encontra regras e correlações entre itens (exemplo: carrinhos de compras).

O aprendizado não supervisionado é muito utilizado em situações em que os dados não possuem rótulos definidos, e o objetivo é descobrir padrões, estruturas ou relações ocultas dentro desses dados. Entre as aplicações mais comuns, destaca-se a segmentação de clientes no marketing, em que algoritmos de agrupamento (como o k-means) são utilizados para identificar grupos de consumidores com comportamentos de compra semelhantes. Essa segmentação permite que empresas personalizem campanhas de forma mais eficaz e aumentem a taxa de conversão.

Outra aplicação relevante é o agrupamento automático de documentos ou notícias por assunto, útil em sistemas de recomendação, organização de conteúdo e motores de busca. Nesses casos, o algoritmo agrupa textos com base em similaridade semântica, facilitando o acesso e a categorização de grandes volumes de informação.

No setor financeiro, o aprendizado não supervisionado também é aplicado na deteção de fraudes em cartões de crédito, ao identificar transações que fogem ao padrão usual do cliente. Como esses sistemas não dependem de exemplos rotulados de fraudes, eles são capazes de detectar anomalias em tempo real, mesmo que nunca tenham visto uma fraude semelhante antes.

Por fim, na área da bioinformática, algoritmos não supervisionados são utilizados para o agrupamento de genes com padrões de expressão semelhantes, o que auxilia na identificação de funções genéticas, diagnóstico de doenças e desenvolvimento de terapias personalizadas. Esses exemplos ilustram como o aprendizado não supervisionado é uma ferramenta poderosa para explorar e estruturar dados complexos, mesmo na ausência de classificações previamente definidas.

Quadro 11 – Principais algoritmos

Algoritmo	Descrição
K-means	Agrupa dados em k clusters baseados em distância
DBSCAN	Agrupa com base na densidade dos dados
PCA (análise de componentes principais)	Redução de dimensionalidade
Autoencoders	Redes neurais usadas para compressão e reconstrução de dados

Já o aprendizado por reforço (reinforcement learning) é um paradigma no qual um agente aprende por meio da interação com um ambiente, tomando decisões que maximizem uma recompensa cumulativa ao longo do tempo. O agente não recebe respostas diretas, mas recompensas ou penalidades com base em suas ações, ajustando seu comportamento para atingir os melhores resultados. Os componentes do aprendizado por reforço são:

- **Agente:** quem toma as decisões (exemplos: um robô, um software).
- **Ambiente:** o contexto onde o agente atua.

- **Estado (state):** a situação atual do agente no ambiente.
- **Ação (action):** uma escolha feita pelo agente.
- **Recompensa (reward):** feedback recebido após uma ação.
- **Política (policy):** estratégia que define as ações do agente.
- **Função de valor (value function):** estima a recompensa futura esperada.

Um exemplo clássico de aprendizado por reforço pode ser representado por um robô que precisa encontrar a saída de um labirinto. No início, o robô não conhece o ambiente nem a localização da saída, mas, a cada ação que realiza, como mover-se para frente, para os lados ou bater em uma parede, ele recebe um sinal de recompensa que indica se aquela ação foi boa ou ruim. Por exemplo, ao avançar corretamente em direção à saída, ele recebe +1 ponto; ao colidir com uma parede, recebe -1 ponto, e ao chegar à saída com sucesso, ganha uma recompensa maior, como +10 pontos.

Com base nessas interações, o robô começa a explorar diferentes caminhos e, ao longo do tempo, aprende a tomar decisões que maximizam sua recompensa total. Esse processo leva à definição de uma política ideal, isto é, uma estratégia que indica qual ação deve ser tomada em cada situação para alcançar o melhor resultado possível.

Esse tipo de aprendizado é muito utilizado em robótica, jogos, sistemas autônomos e otimização de processos, demonstrando grande eficiência em cenários nos quais o agente precisa aprender com a experiência, sem supervisão direta.

Quadro 12 – Principais algoritmos

Algoritmo	Descrição
Q-Learning	Aprende a melhor ação para cada estado usando uma tabela de recompensas
Sarsa	Similar ao Q-Learning, mas aprende com a política seguida
Deep Q-Networks (DQN)	Utiliza redes neurais para representar Q-values em ambientes complexos
Policy gradient	Ajusta diretamente a política de decisões com base no desempenho

O aprendizado por reforço tem ganhado destaque em diversas áreas por sua capacidade de ensinar agentes a tomar decisões sequenciais com base na experiência. Uma das aplicações mais conhecidas está no universo dos jogos, como nos casos emblemáticos do AlphaGo, que superou campeões humanos no jogo de Go, e em jogos eletrônicos complexos, como Dota 2 e xadrez, nos quais agentes de IA aprenderam estratégias sofisticadas por meio da repetição e da maximização de recompensas ao longo do tempo.

Na robótica, o aprendizado por reforço é utilizado para controlar braços mecânicos, permitindo que os robôs aprendam sozinhos a manipular objetos, ajustar movimentos com precisão e se adaptar a diferentes tarefas ou ambientes, mesmo sem programação explícita para cada situação. Outra aplicação relevante está nos sistemas de recomendação personalizados, em que a IA ajusta

continuamente suas sugestões com base nas interações do usuário, buscando oferecer opções que aumentem o engajamento ou a satisfação ao longo do tempo.

O método também é usado na otimização de processos industriais, como no gerenciamento inteligente de energia, em que o sistema aprende a distribuir recursos de forma mais eficiente, reduzindo desperdícios e maximizando o desempenho das operações.

Por fim, na área de navegação autônoma, especialmente em veículos autônomos, o aprendizado por reforço permite que o sistema aprenda a tomar decisões de direção, como frear, acelerar ou desviar de obstáculos, com base em recompensas associadas à segurança e eficiência do trajeto.

Essas aplicações demonstram o enorme potencial do aprendizado por reforço em contextos nos quais o agente precisa aprender a agir de maneira inteligente em ambientes complexos e dinâmicos, adaptando seu comportamento com base nas consequências de suas ações.

Quadro 13 – Comparação entre os tipos de aprendizado

Característica	Supervisionado	Não supervisionado	Por reforço
Dados com rótulo	Sim	Não	Parcial (recompensas)
Tipo de saída	Classe ou valor	Estrutura oculta	Política ótima
Objetivo	Prever ou classificar	Descobrir padrões	Maximizar recompensa
Exemplo típico	Diagnóstico médico	Segmentação de clientes	Jogador de videogame autônomo
Interação com ambiente	Não	Não	Sim

Os paradigmas de aprendizado não supervisionado e reforçado ampliam significativamente a capacidade da IA em lidar com situações em que não há dados rotulados ou onde é necessário tomar decisões em tempo real. Enquanto o não supervisionado, destaca-se por encontrar padrões e estruturas ocultas, o aprendizado por reforço permite que sistemas aprendam por experiência, com base em tentativa e erro.

4.1 Algoritmos de clustering

Clustering é uma técnica fundamental do aprendizado não supervisionado que tem como objetivo principal agrupar dados semelhantes em subconjuntos, chamados de clusters. Nesse processo, os algoritmos analisam as características dos dados e organizam automaticamente os elementos, de forma que os itens dentro de um mesmo grupo sejam mais parecidos entre si do que em relação aos itens de outros grupos.

A semelhança entre os dados é avaliada com base em métricas de distância, sendo a distância euclidiana uma das mais comuns. Essa métrica permite quantificar o quão próximos ou distantes os pontos estão em um espaço multidimensional.

Ao final do processo, os grupos não são definidos previamente, mas sim descobertos pelo próprio algoritmo, tornando o clustering uma abordagem poderosa para identificar padrões ocultos em conjuntos de dados complexos, sendo muito utilizada em áreas como análise de mercado, segmentação de clientes, bioinformática e organização de documentos.

Os principais algoritmos de clustering são:

- **K-means:** é um algoritmo simples e muito utilizado que agrupa os dados em k clusters definidos previamente pelo usuário. Ele funciona atribuindo pontos aos centroides mais próximos e recalculando esses centroides com base na média dos pontos do grupo. É rápido e eficiente em grandes volumes de dados, mas sensível a outliers e exige que os clusters tenham formato esférico e tamanho semelhante.
- **DBSCAN:** é um algoritmo baseado em densidade, que forma clusters ao identificar regiões com alta concentração de pontos. Ele não exige a definição do número de grupos e é robusto a ruídos e outliers, mas pode apresentar dificuldades em conjuntos de dados com densidades muito variadas e depende da escolha adequada dos parâmetros de distância (ϵ) e do número mínimo de pontos (minPts).
- **Clustering hierárquico:** organiza os dados em uma estrutura em árvore (dendrograma), representando a fusão ou divisão dos grupos ao longo do tempo. Pode ser aglomerativo (bottom-up) ou divisivo (top-down), e é útil quando se deseja uma visualização clara das relações entre os dados, embora tenha alto custo computacional e seja sensível a ruídos.
- **Mean shift:** é um algoritmo que identifica clusters com base nos picos de densidade dos dados, movendo iterativamente os pontos em direção às regiões mais densas. Ele não exige definir o número de clusters e lida bem com formas complexas, mas é computacionalmente custoso e sua performance depende da escolha correta da largura da janela (bandwidth).
- **Gaussian mixture models (GMM):** assumem que os dados são gerados por uma combinação de distribuições gaussianas, permitindo que cada ponto tenha uma probabilidade de pertencer a vários grupos. São mais flexíveis que o k-means, especialmente para clusters com sobreposição, mas mais complexos e sensíveis à escolha do número de componentes.

4.1.1 K-means e algoritmos de agrupamento

Entre os diversos algoritmos de agrupamento existentes, o k-means destaca-se como um dos mais populares e utilizados, devido à sua simplicidade, velocidade e boa performance em muitos cenários práticos. No entanto, ele é apenas uma das abordagens disponíveis. Outros algoritmos, como DBSCAN, clustering hierárquico, mean shift e GMM, oferecem alternativas mais robustas para diferentes tipos de distribuição de dados, presença de ruídos ou necessidade de clusters com formatos complexos.

Desse modo, exploraremos agora o funcionamento, as vantagens, as limitações e as aplicações desses algoritmos, com destaque especial para o k-means como ponto de partida para a compreensão das demais abordagens.

K-means

O k-means é um dos algoritmos de clustering mais populares e utilizados no aprendizado não supervisionado, devido à sua simplicidade e eficiência. Seu funcionamento baseia-se na divisão de um conjunto de dados em k grupos distintos, onde k representa o número de clusters que o usuário deve definir previamente. O processo começa com a inicialização de k centroides, que são pontos centrais provisórios de cada grupo. Em seguida, o algoritmo atribui cada ponto de dado ao cluster cujo centroide está mais próximo, geralmente com base na distância euclidiana.

Após essa etapa, os centroides são recalculados com base na média dos pontos que pertencem a cada grupo. Esse ciclo de atribuição e atualização repete-se até que os centroides converjam, ou seja, deixem de mudar significativamente entre as iterações. O k-means é eficaz em grandes volumes de dados, sendo conhecido por sua rapidez e simplicidade de implementação.

Apesar de suas vantagens, o algoritmo apresenta algumas limitações. Um dos principais desafios é a necessidade de definir previamente o número de clusters (k), o que pode não ser trivial em muitos contextos. Além disso, ele é sensível à presença de outliers e à posição inicial dos centroides, o que pode levar a agrupamentos inconsistentes.

Outra limitação é que o algoritmo funciona melhor com clusters de formato esférico e de tamanhos semelhantes, o que pode reduzir sua eficácia em conjuntos de dados mais complexos ou com distribuição irregular.

Ainda assim, o k-means continua sendo uma ferramenta poderosa e frequentemente adotada em aplicações práticas, como segmentação de mercado, compressão de imagens e análise exploratória de dados, especialmente quando se busca uma solução rápida e interpretável.

A seguir, temos um exemplo simples de agrupamento de como usar o k-means em Python utilizando a biblioteca scikit-learn. Nesse exemplo (figura 27 a seguir) estamos gerando dados fictícios usando `make_blobs` da biblioteca scikit-learn, criando um objeto k-means com 4 clusters, treinando o modelo, obtendo os centros dos clusters e os rótulos para cada ponto e, finalmente, plotando os dados e os centros dos clusters no gráfico (figura 28 a seguir).


```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from sklearn.cluster import KMeans
4  from sklearn.datasets import make_blobs
5
6  # Gerar dados fictícios
7  X, _ = make_blobs(n_samples=300, centers=4, random_state=42)
8
9  # Criar um objeto KMeans com 4 clusters
10 kmeans = KMeans(n_clusters=4)
11
12 # Treinar o modelo KMeans
13 kmeans.fit(X)
14
15 # Obter os centros dos clusters e os rótulos para cada ponto
16 centroids = kmeans.cluster_centers_
17 labels = kmeans.labels_
18
19 # Plotar os dados e os centros dos clusters
20 plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
21 plt.scatter(centroids[:, 0], centroids[:, 1], marker='x', s=200, linewidths=3, color='r')
22 plt.xlabel("Feature 1")
23 plt.ylabel("Feature 2")
24 plt.title("K-Means Clustering")
25 plt.show()

```

Figura 27

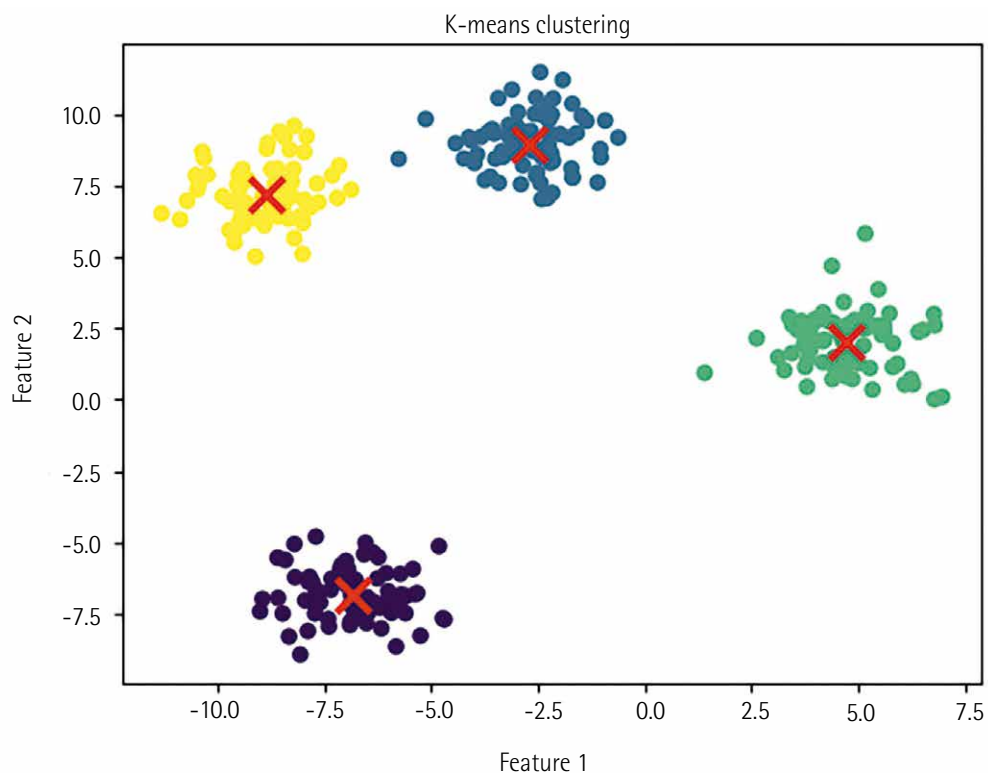


Figura 28



Observação

A função `make_blobs` da biblioteca `scikit-learn` é utilizada para gerar conjuntos de dados sintéticos, especialmente para experimentos e testes com algoritmos de clustering. Ela cria pontos de dados distribuídos em torno de centros definidos aleatoriamente ou especificados pelo usuário, simulando agrupamentos bem separados em um espaço n -dimensional. Os parâmetros da função permitem configurar o número de amostras, de clusters (centros), o desvio padrão de cada grupo, a dimensionalidade dos dados e até o grau de aleatoriedade.

Density-Based Spatial Clustering of Applications with Nois (DBSCAN)

O DBSCAN é um algoritmo de clustering baseado em densidade, projetado para identificar grupos de pontos que estão espacialmente próximos entre si em um conjunto de dados. Ao contrário de métodos como o k -means, o DBSCAN não exige que o número de clusters (k) seja definido previamente. Seu funcionamento depende de dois parâmetros principais: uma distância mínima (ϵ), que define o raio de vizinhança de um ponto, e um número mínimo de pontos (`minPts`) necessários dentro desse raio para que um ponto seja considerado parte de um cluster.

O algoritmo começa selecionando pontos que possuem vizinhos suficientes dentro do raio ϵ . Esses pontos são tratados como núcleos de densidade e formam o início de um cluster. Pontos que estão próximos desses núcleos são agregados ao grupo, e o processo continua até que todos os pontos densamente conectados estejam agrupados. Pontos que não atendem aos critérios de densidade são classificados como ruídos ou outliers, sendo ignorados na formação dos clusters principais.

Entre suas principais vantagens, o DBSCAN se destaca por detectar clusters com formatos arbitrários, como regiões curvas ou irregulares, e por ser robusto à presença de outliers, o que o torna ideal para cenários em que há dados dispersos ou anomalias. Além disso, não exige a definição prévia do número de clusters, o que o torna mais flexível que algoritmos como o k -means.

Por outro lado, o DBSCAN também apresenta desafios. A escolha adequada dos parâmetros ϵ e `minPts` pode ser difícil, especialmente em conjuntos de dados com estruturas complexas. Além disso, seu desempenho pode ser limitado em bases de densidades muito variáveis, já que o algoritmo assume que todos os clusters têm densidade relativamente semelhante.

Mesmo com essas limitações, o DBSCAN é muito utilizado em áreas como análise de dados geoespaciais, detecção de anomalias e reconhecimento de padrões, especialmente quando a forma dos grupos não é conhecida de antemão ou quando a presença de ruídos precisa ser considerada no modelo.

Clustering hierárquico (hierarchical clustering)

O clustering hierárquico é uma técnica de agrupamento que organiza os dados em uma estrutura em forma de árvore, chamada dendrograma, a qual representa o processo de fusão ou divisão dos clusters ao longo do tempo. Essa abordagem permite visualizar de maneira intuitiva como os grupos de dados são formados em diferentes níveis de granularidade. O algoritmo pode operar de duas maneiras principais: de modo aglomerativo (bottom-up), começando com cada ponto como um cluster individual e unindo os mais próximos iterativamente, ou modo divisivo (top-down), iniciando com todos os dados em um único cluster e dividindo-os progressivamente.

Uma das principais vantagens do clustering hierárquico é que ele não exige a definição prévia do número de clusters (k). Em vez disso, o usuário pode analisar o dendrograma e escolher o número de grupos com base no nível de corte mais apropriado. Essa representação gráfica torna o método visual e interpretável, o que é útil para análises exploratórias de dados.

No entanto, o algoritmo apresenta algumas limitações. Seu alto custo computacional o torna menos eficiente para grandes conjuntos de dados, especialmente em abordagens aglomerativas, que exigem o cálculo contínuo de distâncias entre todos os pares de pontos ou clusters. Além disso, o método é sensível a ruídos e outliers, que podem afetar significativamente a estrutura da árvore gerada e comprometer a qualidade dos agrupamentos.

Apesar dessas desvantagens, o clustering hierárquico é muito utilizado em contextos em que se deseja uma visão hierárquica dos dados, como em biologia (classificação de espécies), análise de textos, segmentação de clientes e estudos genéticos, sendo uma ferramenta poderosa para entender relações estruturais entre os elementos de um conjunto de dados.

Mean shift

É um algoritmo de clustering que identifica agrupamentos com base na densidade dos dados, buscando os modos (pontos de maior concentração) no espaço de características. Ao contrário de métodos que dependem da definição prévia do número de clusters, o mean shift não exige que o número de grupos seja informado antecipadamente, tornando-o especialmente útil em análises exploratórias.

Seu funcionamento baseia-se no cálculo de uma função de densidade ao redor dos pontos de dados, utilizando uma janela de análise chamada bandwidth (largura de janela). Cada ponto é deslocado iterativamente em direção às regiões de maior densidade, como se estivesse "subindo" uma colina em um terreno, até convergir para um pico local da distribuição. Os pontos que convergem para o mesmo pico são agrupados no mesmo cluster, o que permite a formação de grupos com formatos arbitrários e complexos, diferentemente de métodos como o k-means, que assumem formas esféricas.

Entre as vantagens do mean shift, destacam-se sua capacidade de lidar com clusters de modos variados e a não necessidade de especificar o número de agrupamentos, o que confere mais flexibilidade ao modelo. No entanto, o algoritmo possui alto custo computacional, principalmente em grandes conjuntos de dados, e sua performance pode ser fortemente influenciada pela escolha da largura

da janela (bandwidth). Um valor inadequado pode levar à formação de poucos ou muitos clusters, comprometendo a qualidade do resultado.

Apesar dessas limitações, o mean shift é uma abordagem poderosa em contextos em que a estrutura dos dados é complexa e desconhecida, sendo aplicado em áreas como visão computacional, rastreamento de objetos e segmentação de imagens.

Algoritmos baseados em modelos (exemplo: Modelo de Mistura Gaussiana – GMM)

Os algoritmos de clustering baseados em modelos, como os Modelos de Mistura Gaussiana (Gaussian Mixture Models – GMM), partem do princípio de que os dados foram gerados por uma combinação de distribuições estatísticas, geralmente gaussianas. Em vez de atribuir rigidamente cada ponto a um único cluster, como no k-means, os GMMs utilizam uma abordagem probabilística, em que cada ponto tem uma probabilidade de pertencer a cada grupo, permitindo representar clusters com sobreposição.

O funcionamento do GMM envolve a utilização do algoritmo expectation-maximization (EM), que itera entre duas fases: a expectation (E-step), que calcula a probabilidade de cada ponto pertencer a cada distribuição, e a maximization (M-step), que ajusta os parâmetros das distribuições (como média, variância e covariância) para melhor se adequar aos dados observados. Esse processo se repete até a convergência, resultando em um modelo que descreve os dados como uma mistura de várias distribuições gaussianas.

Entre suas principais vantagens, destaca-se a flexibilidade do GMM em comparação com algoritmos mais simples, como o k-means. Ele leva em consideração a forma, o tamanho e a orientação dos clusters, o que o torna mais adequado para representar agrupamentos elípticos ou com densidades variadas. Além disso, sua natureza probabilística permite uma análise mais rica dos dados, útil em contextos como reconhecimento de padrões e modelagem estatística.

Por outro lado, o GMM é mais complexo computacionalmente, tanto em termos de implementação quanto de tempo de execução, e está sujeito a superestimar o número de clusters, especialmente se os dados apresentarem ruído ou estruturas não gaussianas. Ainda assim, é uma ferramenta poderosa e utilizada em aplicações que exigem modelagem estatística precisa, como em processamento de sinais, análise de voz, detecção de anomalias e segmentação de imagens.

Quadro 14 – Comparação entre os algoritmos

Algoritmo	Precisa de k?	Lida com outliers?	Formato dos clusters	Tempo de execução
K-means	Sim	Não	Esféricos	Rápido
DBSCAN	Não	Sim	Arbitrário	Médio
Hierárquico	Não	Não	Arbitrário	Lento
Mean shift	Não	Parcialmente	Arbitrário	Alto
GMM	Sim	Parcialmente	Elípticos	Médio

Aplicações práticas de clustering

O clustering, ou agrupamento, é uma técnica de aprendizado não supervisionado com aplicação prática em diversos setores, graças à sua capacidade de descobrir padrões e organizar dados sem a necessidade de rótulos prévios. No marketing, por exemplo, o clustering é utilizado para a segmentação de clientes com base em comportamentos de compra, preferências ou perfis de consumo, permitindo que as empresas personalizem campanhas e otimizem suas estratégias de relacionamento com o cliente.

Na área de ciência de dados, o clustering é empregado no agrupamento de documentos, imagens e perfis de usuários, auxiliando em sistemas de recomendação, motores de busca e organização automática de grandes volumes de informação. Já na biologia, essa técnica é aplicada na classificação de genes com padrões de expressão semelhantes, sendo uma ferramenta valiosa em pesquisas genéticas, estudos de doenças e desenvolvimento de tratamentos personalizados.

No campo da segurança, o clustering é útil na detecção de padrões incomuns em dados, como em análises forenses digitais, podendo ajudar a identificar comportamentos anômalos, fraudes ou acessos suspeitos. Na educação, a técnica pode ser aplicada para agrupar estudantes com estilos de aprendizagem semelhantes, possibilitando a personalização do ensino e o desenvolvimento de estratégias pedagógicas mais eficazes.

Essas aplicações demonstram a versatilidade e o valor do clustering como uma ferramenta essencial para análise exploratória de dados, descoberta de conhecimento e tomada de decisões mais informadas em contextos variados.

Para a visualização dos resultados, é comum utilizar técnicas de redução de dimensionalidade, como PCA ou t-SNE, para representar visualmente os clusters em 2D ou 3D e facilitar a interpretação.

Os algoritmos de clustering são ferramentas poderosas do aprendizado não supervisionado, permitindo a descoberta de padrões ocultos e agrupamentos naturais em conjuntos de dados. A escolha do algoritmo mais adequado depende das características do problema, como o formato dos dados, a presença de ruídos e o conhecimento prévio do número de clusters. O domínio dessas técnicas permite ao desenvolvedor explorar dados de maneira mais inteligente, contribuindo para análises, recomendações e decisões automatizadas.

4.1.2 Aplicações de agrupamento em IA

O agrupamento (ou clustering) é uma técnica valiosa no contexto da análise de dados porque permite identificar padrões e estruturas naturais dentro de grandes conjuntos de informações, mesmo sem a presença de rótulos. Ao organizar os dados em grupos com características semelhantes, o clustering facilita a visualização e a interpretação, tornando mais compreensível o comportamento coletivo dos dados e revelando tendências que poderiam passar despercebidas em análises individuais.

Além disso, o agrupamento reduz a complexidade de tarefas posteriores, servindo como etapa preliminar para problemas de classificação e regressão, ao estruturar os dados de maneira mais organizada. Essa técnica também é fundamental para segmentação e personalização em áreas como marketing, saúde, segurança e educação, permitindo tomadas de decisão automatizadas com base em

perfis ou padrões identificados. Por essas razões, o clustering se destaca como uma ferramenta essencial para a descoberta de conhecimento e apoio à decisão em contextos variados.

A seguir, serão apresentadas as principais aplicações de agrupamento por área.

Marketing e varejo

No setor de marketing e varejo, os algoritmos de agrupamento são utilizados para segmentar clientes com base em seu comportamento de compra, levando em consideração variáveis como frequência de compras, ticket médio, preferências e histórico de navegação. Essa segmentação permite que as empresas entendam melhor os diferentes perfis de consumidores e desenvolvam estratégias personalizadas para cada grupo, como campanhas específicas para clientes fiéis, novos consumidores ou clientes inativos.

Além disso, o clustering possibilita a recomendação direcionada de produtos a grupos com gostos semelhantes, algo muito comum em plataformas de e-commerce, em que sistemas inteligentes sugerem itens com base nos padrões identificados em consumidores com perfis parecidos. Um exemplo prático é o de um supermercado on-line que utiliza o algoritmo k-means para identificar grupos de clientes e, com base nisso, oferece promoções personalizadas, como descontos em produtos frequentemente comprados por determinado segmento. Essa abordagem aumenta a eficácia das ações de marketing, melhora a experiência do cliente e contribui para o aumento das vendas e da fidelização.

Saúde e biomedicina

Na área da saúde e biomedicina, os algoritmos de agrupamento auxiliam na descoberta de padrões clínicos e biológicos a partir de grandes volumes de dados. Uma aplicação comum é o agrupamento de pacientes com base em sintomas, respostas a tratamentos ou histórico médico, permitindo identificar perfis clínicos semelhantes que podem auxiliar na definição de protocolos personalizados e mais eficazes. O clustering também é utilizado em análises genéticas, como na identificação de genes com padrões de expressão semelhantes, o que pode revelar relações funcionais entre genes e ajudar na compreensão de doenças complexas.

Outra aplicação relevante está na detecção de surtos ou epidemias, por meio do agrupamento de casos com sintomas e localizações geográficas semelhantes, o que permite respostas mais rápidas e direcionadas por parte das autoridades de saúde. Um exemplo prático é o de pesquisadores que utilizam técnicas de clustering para identificar subtipos de uma doença rara com base em dados laboratoriais e de exames clínicos. Essas abordagens contribuem não apenas para diagnósticos mais precisos, mas também para avanços na medicina personalizada e na saúde pública.

Segurança da informação

Na área de segurança da informação, os algoritmos de clustering são utilizados como ferramentas poderosas para detecção de anomalias e comportamentos fora do padrão em redes e sistemas computacionais. Uma das aplicações mais comuns é a identificação de intrusões ou tráfego incomum, em que os dados de rede são agrupados com base em suas características e fluxos, permitindo destacar padrões suspeitos ou divergentes da atividade normal. O clustering também é útil na análise forense

digital, permitindo o agrupamento de arquivos, logs ou eventos com base em similaridade, o que auxilia na investigação de incidentes e na reconstrução de ações maliciosas.

Outra aplicação relevante é a identificação de usuários que apresentam comportamentos atípicos em ambientes corporativos, como acessos em horários incomuns, uso anormal de recursos ou manipulação de arquivos confidenciais. Por exemplo, um sistema de segurança baseado em clustering pode detectar acessos fora do padrão – como uma sequência de logins em horários não usuais – e sinalizar esses eventos para análise. Essas técnicas fortalecem as estratégias de prevenção e resposta a incidentes, proporcionando mais inteligência e agilidade na proteção de dados e infraestruturas críticas.

Processamento de linguagem natural (PLN)

No campo do PLN, os algoritmos de clustering são utilizados para organizar e interpretar grandes volumes de texto sem a necessidade de rótulos manuais. Uma aplicação comum é o agrupamento automático de documentos, artigos ou notícias por temas, permitindo que sistemas classifiquem conteúdos semelhantes com base na análise do vocabulário e da estrutura linguística. Além disso, técnicas de clustering são empregadas na análise de sentimentos, agrupando opiniões semelhantes, o que é útil para identificar tendências em avaliações de produtos, comentários em redes sociais ou respostas de pesquisas.

Outra aplicação relevante é o agrupamento de conversas em chats ou interações em redes sociais, com o objetivo de identificar tópicos recorrentes ou intenções de comunicação, como reclamações, elogios ou dúvidas. Um exemplo prático é o de um portal de notícias que utiliza o algoritmo DBSCAN para organizar automaticamente os artigos por assunto, sem a necessidade de rotulação prévia. Esse tipo de abordagem otimiza o acesso à informação, melhora a navegação por conteúdo e apoia sistemas de recomendação e categorização automatizada de textos.

Educação

Na área da educação, os algoritmos de clustering têm se mostrado valiosos para compreender melhor o perfil dos estudantes e apoiar a personalização do ensino. Uma das aplicações mais relevantes é a identificação de estilos de aprendizagem, permitindo agrupar alunos com preferências semelhantes – como aqueles que aprendem melhor por meio de recursos visuais, auditivos ou práticos – e, assim, adaptar os métodos de ensino de modo mais eficaz. O clustering também é utilizado para agrupar estudantes com desempenho acadêmico parecido, facilitando a aplicação de intervenções pedagógicas direcionadas, como tutorias, reforços ou desafios extras, conforme as necessidades de cada grupo.

Outra aplicação importante é a análise de evasão escolar, em que os dados de frequência, participação, notas e uso da plataforma são agrupados para identificar padrões de comportamento que indicam risco de abandono. Um exemplo prático disso é uma plataforma de ensino que utiliza algoritmos de clustering para agrupar alunos com base em seu desempenho e hábitos de estudo, oferecendo trilhas de aprendizagem personalizadas e aumentando o engajamento. Essas estratégias tornam o processo educacional mais adaptável, eficiente e centrado nas necessidades individuais dos estudantes.

Transporte e mobilidade

No setor de transporte e mobilidade, os algoritmos de clustering auxiliam na otimização de trajetos, alocação de recursos e melhoria da eficiência logística. Uma aplicação comum é o agrupamento de rotas semelhantes, o que permite identificar trajetos redundantes ou sobrepostos e, com isso, reduzir custos operacionais e tempo de deslocamento. Além disso, dados de GPS coletados de veículos ou dispositivos móveis podem ser utilizados para identificar zonas de congestionamento ao agrupar áreas com alta densidade de tráfego, contribuindo para o planejamento de vias e estratégias de mobilidade urbana.

Outra aplicação relevante é o planejamento de transporte público com base na demanda agrupada, permitindo ajustar horários, itinerários e número de veículos de acordo com os padrões de uso da população em diferentes regiões e horários. Um exemplo prático é o de um aplicativo de mobilidade urbana que utiliza técnicas de clustering para analisar a concentração de usuários em tempo real, adaptando automaticamente o número de veículos disponíveis em determinadas áreas. Com isso, é possível melhorar a qualidade do serviço, reduzir o tempo de espera e aumentar a satisfação dos usuários, promovendo uma mobilidade mais inteligente e eficiente.

Entretenimento e mídia

No setor de entretenimento e mídia, os algoritmos de clustering são utilizados para entender os hábitos de consumo dos usuários e oferecer experiências personalizadas. Uma das aplicações mais comuns é o agrupamento de usuários com base em seu perfil de consumo, como preferências por determinados gêneros de filmes, séries, músicas ou estilos de conteúdo. Ao identificar grupos com gostos semelhantes, as plataformas podem oferecer recomendações mais assertivas e relevantes, aumentando o engajamento e a satisfação dos usuários.

Além disso, o clustering é utilizado na geração automática de playlists e listas de recomendação, com base no comportamento coletivo de grupos semelhantes, otimizando o tempo de curadoria manual e oferecendo conteúdos mais alinhados com os interesses do público. Um exemplo prático é o de um serviço de streaming que identifica usuários que assistem com frequência a thrillers, agrupando-os e, a partir disso, recomendando séries e filmes semelhantes a todos os membros do grupo. Esse tipo de abordagem promove uma experiência de consumo mais personalizada e eficiente, beneficiando tanto os usuários quanto as plataformas.



Lembrete

As aplicações de agrupamento em inteligência artificial abrangem diversos setores e são fundamentais para descobrir padrões, entender o comportamento dos dados e tomar decisões inteligentes sem a necessidade de rótulos manuais. Seja para segmentar clientes, identificar anomalias ou personalizar serviços, os algoritmos de clustering – como K-means, DBSCAN e hierárquico – oferecem poderosas ferramentas para transformar dados brutos em conhecimento útil.

4.2 Aprendizado por reforço

O aprendizado por reforço (RL) é um dos ramos mais fascinantes da inteligência artificial, pois simula o processo de aprendizado por tentativa e erro, semelhante ao que ocorre com seres humanos e animais. Nessa abordagem, um agente aprende a tomar decisões interagindo com um ambiente, visando maximizar recompensas ao longo do tempo.

O aprendizado por reforço é um modelo baseado em ações, experiências e recompensas. Um agente inteligente explora um ambiente, toma decisões e recebe feedback na forma de recompensas ou penalidades. Com o tempo, ele ajusta sua política de decisão para escolher ações que tragam os melhores resultados.

Quadro 15 – Componentes fundamentais do aprendizado por reforço

Elemento	Descrição
Agente	Quem aprende e toma decisões (exemplo: robô, software)
Ambiente	Onde o agente atua (exemplo: jogo, sistema, simulação)
Estado (s)	Situação atual do ambiente
Ação (a)	Escolha feita pelo agente
Recompensa (r)	Valor numérico recebido após uma ação
Política (π)	Estratégia de decisão do agente
Função de valor (V(s))	Mede a expectativa de recompensa em um estado
Função Q (Q(s, a))	Mede o valor esperado de executar uma ação a em um estado s

Funcionamento básico:

- o agente observa o estado atual do ambiente;
- escolhe uma ação com base em sua política atual;
- o ambiente responde com um novo estado e uma recompensa;
- o agente atualiza sua estratégia com base na recompensa recebida;
- o ciclo se repete, e o agente aprende com a experiência acumulada.

4.2.1 Algoritmos de aprendizado por reforço

Q-Learning

O Q-Learning é um dos algoritmos mais conhecidos no contexto do aprendizado por reforço, caracterizado por ser baseado em tabela e por permitir que um agente aprenda a tomar decisões de forma ótima ao interagir com um ambiente. Seu objetivo é estimar a função de valor $Q(s, a)$, que representa a

qualidade de realizar uma determinada ação em um estado s , indicando o quanto de recompensa futura o agente pode esperar ao seguir a melhor política a partir desse ponto.

A atualização dos valores da função Q é feita por meio de uma fórmula que considera tanto a recompensa imediata quanto o valor estimado do melhor cenário futuro. A equação é dada por:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma * \max_{a'} Q(s', a') - Q(s, a)]$$

Onde:

- α é a taxa de aprendizado, controlando o quanto o novo valor influencia o valor atual;
- γ é o fator de desconto, que determina o peso das recompensas futuras;
- s' representa o próximo estado alcançado após a ação;
- a' é a ação futura possível nesse novo estado.

Ao longo do tempo e das interações com o ambiente, o algoritmo ajusta os valores da tabela Q , permitindo que o agente aprenda uma política ótima que maximize a recompensa acumulada.

- **SARSA (State-Action-Reward-State-Action):** é similar ao Q-Learning, mas atualiza a função Q com base na ação que realmente foi tomada, e não na melhor possível.
- **Deep Q-Learning (DQN):** usa redes neurais profundas para aproximar a função Q . Ideal para ambientes complexos com muitos estados possíveis.
- **Policy gradient:** em vez de estimar Q , aprende diretamente a política de ações. Muito usado em tarefas contínuas (exemplos: controle de robôs, jogos com múltiplas opções).

No aprendizado por reforço, um dos principais desafios enfrentados pelo agente é o dilema entre exploração e exploração (exploration vs. exploitation). Esse dilema consiste em equilibrar a necessidade de explorar novas ações – que ainda não foram testadas, mas podem levar a recompensas maiores – com a tendência de explorar (ou explotar) o conhecimento já adquirido, ou seja, repetir ações conhecidas que já geraram boas recompensas no passado. Explorar permite que o agente descubra novas estratégias e evite ficar preso a soluções subótimas, enquanto explotar garante o aproveitamento do que já foi aprendido para maximizar o ganho acumulado.

Para lidar com esse equilíbrio de forma eficiente, são utilizadas técnicas como a ϵ -greedy, que introduz aleatoriedade controlada no processo de decisão. Nesse método, o agente escolhe a melhor ação conhecida na maior parte do tempo, mas com uma pequena probabilidade ϵ (epsilon), ele opta por uma ação aleatória, permitindo a exploração de novas possibilidades. À medida que o agente aprende mais sobre o ambiente, o valor de ϵ pode ser reduzido gradualmente, tornando o comportamento mais voltado à exploração. Esse mecanismo é essencial para garantir que o agente aprenda de maneira eficaz e converta para uma política ótima ao longo do tempo.



Observação

Na matemática e na estatística, a letra grega ϵ (épsilon) é comumente utilizada para representar uma quantidade pequena, positiva e arbitrária, muito próxima de zero, mas nunca igual a zero. Em probabilidade, esse símbolo aparece frequentemente em contextos que envolvem limites, aproximações e margens de erro. Um exemplo clássico é na definição de convergência em probabilidade, onde dizemos que uma sequência de variáveis aleatórias X_n converge para uma variável X se, para todo valor de ϵ maior que zero, a probabilidade de que a diferença entre X_n e X seja maior que ϵ tende a zero à medida que n cresce. Ou seja, quanto maior o número de observações, menor a chance de a variável estimada se afastar do valor real por mais que essa pequena margem ϵ .

Além disso, o épsilon é utilizado para expressar níveis de tolerância ou incerteza em algoritmos e modelos. Um bom exemplo é o algoritmo ϵ -greedy, muito usado em aprendizado por reforço, onde ϵ representa a probabilidade de exploração, isto é, a chance de o algoritmo tomar uma decisão aleatória em vez de seguir a melhor estratégia conhecida até o momento. Se ϵ for igual a 0,1, por exemplo, isso indica que o modelo tomará decisões exploratórias em 10% dos casos e usará a melhor ação conhecida nos 90% restantes. Portanto, o valor de ϵ regula o equilíbrio entre explorar novas possibilidades e aproveitar o que já se conhece.

Por fim, em demonstrações matemáticas envolvendo probabilidade, é comum usar ϵ para indicar que certos eventos têm probabilidades tão pequenas, que podem ser ignorados em termos práticos, sendo menores do que qualquer valor ϵ positivo. Em todos esses contextos, o uso do épsilon ajuda a expressar com precisão a ideia de proximidade, erro aceitável ou incerteza mínima, elementos centrais na teoria da probabilidade e em muitas aplicações na ciência de dados.

Quadro 16 – Aplicações práticas do aprendizado por reforço

Área	Aplicação (exemplo)
Jogos	Agentes que jogam xadrez, Go, Atari, Dota 2
Robótica	Controle de braços robóticos, locomoção autônoma
Finanças	Otimização de carteiras, negociação automatizada
Sistemas de recomendação	Adaptação dinâmica de sugestões
Manufatura	Otimização de linhas de produção e alocação de tarefas
Mobilidade urbana	Aprendizado de rotas ideais com base em tráfego

Um exemplo clássico para ilustrar o funcionamento do aprendizado por reforço é o de um robô tentando encontrar a saída de um labirinto. Inicialmente, o robô não possui conhecimento prévio sobre o ambiente, movendo-se de modo exploratório e cometendo erros. A cada vez que colide com uma parede, ele recebe uma recompensa negativa, sinalizando que aquela ação foi indesejada. Por outro lado, ao alcançar a saída do labirinto, o robô recebe uma recompensa positiva, reforçando o comportamento que o levou ao sucesso.

Ao longo de várias tentativas, o robô começa a aprender com a experiência, ajustando suas decisões com base nas recompensas recebidas. Ele passa a evitar caminhos que resultam em punições e prioriza rotas que o aproximam da saída. Com o tempo, desenvolve uma política ideal de navegação, permitindo que chegue à meta de modo cada vez mais rápida e eficiente. Esse processo de tentativa e erro, reforçado por recompensas, é a essência do aprendizado por reforço.

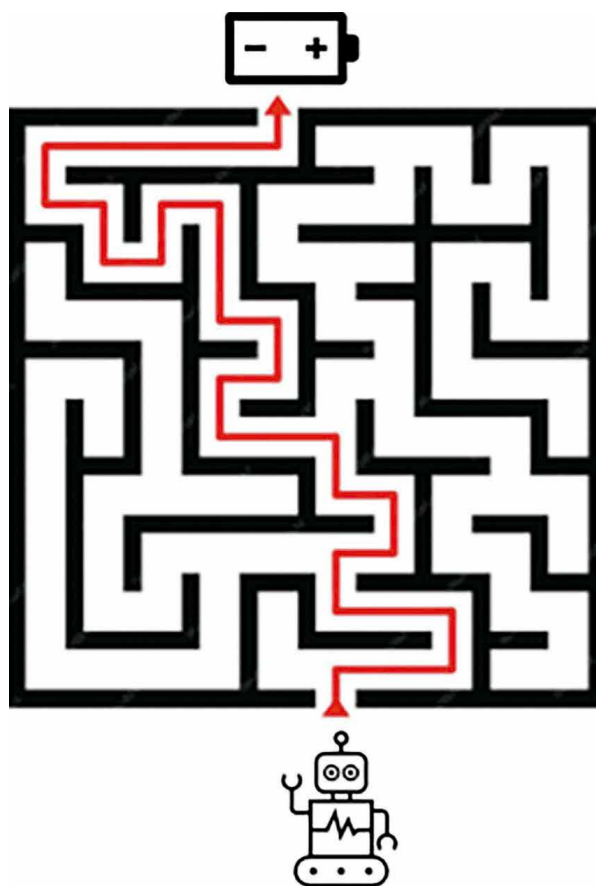


Figura 29 – Robô no labirinto

Embora o aprendizado por reforço seja uma técnica poderosa para tomada de decisões sequenciais, ele também apresenta diversos desafios práticos. Um dos principais obstáculos está relacionado a ambientes com muitos estados e ações possíveis, que exigem um alto poder computacional para que o agente possa explorar adequadamente todas as combinações e aprender uma política eficiente. Em ambientes complexos, o processo de aprendizado pode ser lento e demorado, dificultando a convergência para uma solução ótima.

Além disso, é particularmente difícil treinar agentes em ambientes não determinísticos, nos quais uma mesma ação pode levar a resultados diferentes dependendo do contexto ou do acaso. Nesses casos, o agente precisa aprender a lidar com incertezas e variabilidades, o que torna o aprendizado menos previsível. Outro fator crítico é a modelagem da função de recompensa, que deve ser cuidadosamente projetada para realmente orientar o agente ao comportamento desejado. Recompensas mal definidas podem levar a estratégias ineficazes ou até mesmo a comportamentos indesejados. Por isso, o sucesso do aprendizado por reforço depende não apenas do algoritmo utilizado, mas também de uma boa representação do ambiente e de objetivos bem especificados.

O aprendizado por reforço representa uma abordagem poderosa para o desenvolvimento de agentes inteligentes autônomos, capazes de aprender com a experiência e tomar decisões otimizadas ao longo do tempo. Sua capacidade de lidar com ambientes dinâmicos e interativos torna-o essencial para aplicações avançadas de IA, especialmente em robótica, jogos, finanças e sistemas adaptativos, combinado com redes neurais e técnicas de deep learning.

4.2.2 Conceitos de aprendizado por reforço

O RL é um paradigma de inteligência artificial baseado na interação entre um agente e um ambiente, com o objetivo de aprender a tomar decisões que maximizem recompensas ao longo do tempo. Diferentemente do aprendizado supervisionado e não supervisionado, o aprendizado por reforço envolve ação, feedback e adaptação contínua, sendo muito aplicado em jogos, robótica e sistemas autônomos.

Visão geral do aprendizado por reforço

No RL, um agente aprende por tentativa e erro, executando ações em um ambiente dinâmico. Cada ação gera um novo estado e uma recompensa, que é usada como feedback para ajustar o comportamento do agente. Com o tempo, o agente aprende qual decisão tomar em cada situação para obter o maior benefício possível.

O aprendizado por reforço é uma técnica de IA em que um agente aprende a tomar decisões através da interação com um ambiente, com o objetivo de maximizar uma recompensa cumulativa.

O ciclo básico segue quatro etapas principais:

- **Observação do estado atual:** o agente percebe o estado atual do ambiente. O estado é uma representação da situação naquele momento.
 - **Exemplo:** em um jogo de labirinto, o agente observa sua posição atual no mapa.
- **Execução de uma ação:** com base em sua estratégia (ou política), o agente escolhe uma ação para realizar naquele estado.
 - **Exemplo:** o agente decide mover-se para a direita.

- **Resposta do ambiente (novo estado e recompensa):** o ambiente reage à ação do agente, fornecendo um novo estado, ou seja, a nova situação após a ação; e uma recompensa, que pode ser positiva (ganho), negativa (penalidade) ou neutra.
 - **Exemplo:** o agente bateu em uma parede. O novo estado é a mesma posição, e a recompensa é -1.
- **Atualização da estratégia:** com base no novo estado e na recompensa recebida, o agente ajusta sua política para melhorar suas decisões futuras. Isso é feito por algoritmos como Q-learning ou Deep Q-Networks (DQN).
 - **Exemplo:** o agente aprende que ir para a direita naquele estado não é bom e evita essa ação no futuro.

Exemplo prático: robô no labirinto

Imagine um robô que está aprendendo a sair de um labirinto:

- **Estado atual:** robô está na posição (2,3).
- **Ação:** decide ir para cima.
- **Novo estado:** agora está na posição (1,3).
- **Recompensa:** +10 (porque se aproximou da saída).
- **Atualização:** o robô registra que ir para cima foi uma boa escolha nesse ponto do labirinto.

Com o tempo e com muitas repetições desse ciclo, o agente aprende quais ações são melhores em cada estado para alcançar seu objetivo, neste caso, sair do labirinto o mais rápido possível.

Principais conceitos do aprendizado por reforço

- **Agente:** o agente é o componente responsável por tomar decisões dentro de um ambiente. Ele executa ações com base em uma política de comportamento e aprende por meio da interação contínua com o ambiente, buscando maximizar sua performance.
- **Ambiente:** o ambiente é o sistema externo no qual o agente está inserido e com o qual ele interage. É o ambiente que determina os estados possíveis, as regras de transição entre estados e o modelo de recompensas, ou seja, como as ações do agente geram consequências.
- **Estado (s):** o estado representa a condição atual do ambiente em um dado momento, conforme percebido pelo agente. Pode variar em complexidade: desde algo simples, como a posição de um robô em uma grade, até representações mais complexas, como uma imagem capturada por uma câmera em um jogo.

- **Ação (a):** a ação é a escolha feita pelo agente com base no estado atual. Cada ação influencia o ambiente de alguma forma, levando a uma transição para um novo estado. O conjunto de ações disponíveis depende do contexto e das regras do ambiente.
- **Recompensa (r):** a recompensa é um valor numérico (geralmente um número real) fornecido ao agente após a execução de uma ação. Esse valor serve como um sinal de feedback, indicando se a ação realizada foi benéfica ou prejudicial, de acordo com o objetivo final. O agente busca maximizar a soma total das recompensas acumuladas ao longo do tempo.
- **Política (π):** a política é a estratégia que orienta o comportamento do agente, determinando qual ação deve ser tomada em cada estado. Ela pode ser determinística, quando define uma única ação específica para cada estado, ou estocástica, quando associa diferentes ações a diferentes probabilidades.
- **Função de valor ($V(s)$):** a função de valor estima a utilidade de um estado, ou seja, representa a recompensa esperada que o agente pode obter a partir desse estado, assumindo que seguirá a política atual. É usada para avaliar o quão promissor é estar em determinado estado.
- **Função de ação-valor ($Q(s, a)$):** a função de ação-valor associa um valor a um par (estado, ação), indicando o quanto é vantajoso executar uma determinada ação em um estado específico, considerando a política seguida. Essa função é fundamental em algoritmos como o Q-Learning, pois orienta o agente a escolher as ações com maior retorno esperado.

Modelo de Decisão de Markov (Markov Decision Process – MDP)

O MDP é uma estrutura matemática utilizada para modelar problemas de aprendizado por reforço, em que um agente aprende a tomar decisões em um ambiente incerto e dinâmico. Ele descreve como o agente interage com o ambiente ao longo do tempo, tomando ações, recebendo recompensas e mudando de estado.

Um MDP é definido, formalmente, por cinco elementos principais:

- **Conjunto de estados (S):** representa todas as situações possíveis em que o agente pode se encontrar.
- **Conjunto de ações (A):** é o conjunto de todas as ações que o agente pode realizar. A escolha de uma ação depende do estado atual.
- **Função de transição (T):** é definida como $T(s, a, s')$ e representa a probabilidade de o agente ir do estado s para o estado s' após executar a ação a . Por exemplo, se um robô tenta andar para frente (ação) estando em um piso escorregadio (estado), há 80% de chance de avançar (s'), mas 20% de escorregar e permanecer no mesmo lugar.

- **Função de recompensa (R):** indica o valor numérico (recompensa) que o agente recebe ao executar uma ação em um determinado estado.
- **Política (π):** define a estratégia de comportamento do agente, ou seja, qual ação ele deve escolher em cada estado.

A propriedade de Markov afirma que a próxima decisão depende apenas do estado atual, e não do histórico de estados anteriores. Em outras palavras, o processo não tem memória: tudo o que o agente precisa para decidir sua próxima ação está contido no estado atual. Se um carro autônomo está parado no semáforo vermelho, sua decisão de esperar ou avançar deve se basear apenas no estado atual (semáforo vermelho), e não na sequência de semáforos anteriores que enfrentou.

Exploração vs. exploração

No aprendizado por reforço, o agente precisa aprender qual ação tomar em cada situação para maximizar suas recompensas ao longo do tempo. No entanto, durante esse processo, ele enfrenta um dilema conhecido como exploração vs. exploração (em inglês, exploration vs. exploitation).

- **Exploração (exploration):** refere-se à atitude do agente de experimentar ações novas, mesmo que ele ainda não saiba se elas são boas. O objetivo da exploração é descobrir novas possibilidades, que talvez levem a recompensas maiores no futuro. Exemplo: um robô pode decidir entrar por um caminho desconhecido no labirinto para verificar se ele leva a uma saída mais rápida, mesmo correndo o risco de bater em uma parede.
- **Exploração (exploitation):** refere-se à decisão de usar o conhecimento já adquirido, ou seja, repetir as ações que o agente já sabe que funcionam bem. O objetivo aqui é maximizar a recompensa imediata, com base na experiência passada. Exemplo: o robô já conhece um caminho seguro até a saída do labirinto e escolhe sempre esse trajeto para garantir a recompensa final, mesmo que exista um caminho mais curto ainda desconhecido.

Esse equilíbrio entre explorar e explorar é fundamental no processo de aprendizado, porém há um dilema:

- Se o agente explorar pouco, ele pode nunca descobrir estratégias melhores e se acomodar em soluções subótimas.
- Se o agente explorar demais, ele gasta tempo testando ações ruins e demora para usar as boas que já descobriu.

Por isso, algoritmos de aprendizado por reforço frequentemente usam estratégias para equilibrar esse dilema, como:

- **ϵ -greedy:** o agente escolhe a melhor ação conhecida com alta probabilidade (exploração), mas às vezes escolhe uma ação aleatória (exploração).

- **Softmax:** atribui probabilidades diferentes a cada ação com base no valor estimado, permitindo um equilíbrio gradual.
- **Dcaimento de ϵ :** o agente começa explorando bastante e, ao longo do tempo, vai explorando menos e explorando mais, conforme aprende.

Exemplo de aplicação

Exemplo 1: caça ao tesouro

Imagine um agente jogando um jogo de caça ao tesouro. No início, ele explora diversas trilhas e salas, mesmo sem saber o que encontrará. Ao encontrar uma sala que sempre dá 5 moedas, ele começa a explorar esse caminho. No entanto, pode haver uma sala escondida com 10 moedas, e se ele parar de explorar cedo demais, nunca vai descobri-la.

Assim, fica claro que um bom agente precisa explorar o suficiente para conhecer bem o ambiente, mas explorar o necessário para colher os frutos do que aprendeu.

Exemplo 2: corredor de seis posições (0 a 5)

O agente começa sempre na posição 0. O objetivo é chegar na posição 5 (meta) e ganhar +1 ponto. Ele pode se mover apenas para a direita (ação 1) ou para a esquerda (ação 0). Se tentar sair dos limites, permanece no mesmo lugar.

```

1  import numpy as np
2  import random
3
4  # Parâmetros do ambiente
5  n_states = 6          # posições: 0 a 5
6  n_actions = 2         # 0 = esquerda, 1 = direita
7  goal_state = 5        # objetivo
8
9  # Parâmetros do Q-learning
10 alpha = 0.1           # taxa de aprendizado
11 gamma = 0.9           # fator de desconto
12 epsilon = 0.2         # exploração
13 episodes = 200        # número de episódios de treinamento
14
15 # Inicializa a tabela Q com zeros
16 Q = np.zeros((n_states, n_actions))
17
18 # Função do ambiente: retorna próximo estado, recompensa e se terminou
19 def step(state, action):
20     if action == 0: # esquerda
21         next_state = max(0, state - 1)
22     else:           # direita
23         next_state = min(n_states - 1, state + 1)
24
25     reward = 1 if next_state == goal_state else 0
26     done = next_state == goal_state
27     return next_state, reward, done
28
29 # Treinamento do agente
30 for episode in range(episodes):
31     state = 0
32     done = False
33
34     while not done:
35         # Escolha da ação (ε-greedy)
36         if random.uniform(0, 1) < epsilon:
37             action = random.randint(0, n_actions - 1) # Explorar
38         else:
39             action = np.argmax(Q[state])              # Explorar
40
41         next_state, reward, done = step(state, action)
42
43         # Atualiza a Tabela Q
44         Q[state, action] += alpha * (reward + gamma * np.max(Q[next_state]) - Q[state, action])
45         state = next_state
46
47 # Impressão da Tabela Q aprendida
48 print("Tabela Q após o treinamento:")
49 for s in range(n_states):
50     print(f"Estado {s}: {Q[s]}")
51
52 # Mostra a política aprendida
53 print("Política aprendida (melhor ação por estado):")
54 for state in range(n_states):
55     action = np.argmax(Q[state])
56     acao_texto = '→' if action == 1 else '←'
57     print(f"Posição {state}: {acao_texto}")

```

Figura 30

Análise da saída:

A tabela Q (Q-table) é uma estrutura de dados (geralmente uma matriz), onde cada célula representa o valor esperado de uma ação em um estado. Ela responde à pergunta: "se eu estiver no estado s e escolher a ação a , qual é a recompensa total esperada se eu continuar seguindo a melhor estratégia?".

Na linha 16 da figura 30, temos a inicialização da tabela Q:

```
Q = np.zeros((n_states, n_actions))
```

- **n_states = 6**: o corredor tem seis posições (estados 0 a 5);
- **n_actions = 2**: duas ações possíveis:
 - **0**: mover para a esquerda ($\leftarrow$)
 - **1**: mover para a direita ($\rightarrow$)

Ou seja, a tabela Q tem 6 linhas (uma por posição no corredor) e 2 colunas (uma por ação).

Exemplo 3: simulado

Depois do treinamento, a tabela Q pode ficar assim:

Tabela 1

Estado	$\leftarrow$ (esquerda)	$\rightarrow$ (direita)	Melhor ação
0	0.00	0.68	$\rightarrow$
1	0.45	0.73	$\rightarrow$
2	0.50	0.78	$\rightarrow$
3	0.52	0.82	$\rightarrow$
4	0.53	0.90	$\rightarrow$
5	0.00	0.00	- (meta)

- Note que a coluna tem os maiores valores, indicando que ir para a direita é sempre a melhor ação.
- No estado 5, não há ação melhor porque é o estado terminal.

A tabela Q funciona como uma memória de aprendizado: ela guarda o "conhecimento" do agente sobre o que fazer em cada situação com base na experiência. Quanto mais o agente interage com o ambiente, mais precisa a tabela se torna.

Os conceitos de aprendizado por reforço formam a base para o desenvolvimento de agentes que aprendem com a prática e ajustam suas decisões para maximizar resultados ao longo do tempo. Ao compreender elementos como estados, ações, recompensas, política e funções de valor, é possível projetar sistemas inteligentes que se adaptam e melhoram constantemente por meio da interação com o ambiente. Essa abordagem aproxima cada vez mais a IA de comportamentos inteligentes similares aos humanos.

4.2.3 Aplicação em robótica e jogos

O aprendizado por reforço tem ganhado grande destaque na IA moderna, especialmente em áreas como robótica e jogos digitais, em que a capacidade de tomar decisões em tempo real e aprender com a experiência é essencial. Esses cenários interativos representam ambientes ideais para a aplicação de RL, permitindo que agentes inteligentes desenvolvam estratégias complexas por meio da experimentação.

O uso de RL em robótica e jogos se justifica pelo fato de que, em ambos os contextos, o agente precisa interagir com um ambiente dinâmico, tomar decisões em tempo real, lidar com incertezas e enfrentar múltiplas possibilidades de ação. Além disso, o agente busca constantemente aprimorar seu desempenho com base nas recompensas obtidas durante suas interações. Essas exigências se alinham diretamente aos princípios fundamentais do RL, tornando essa abordagem uma ferramenta poderosa para o desenvolvimento de sistemas autônomos, capazes de aprender, se adaptar e otimizar suas ações em tarefas complexas, seja no controle de robôs ou na execução de estratégias em jogos.

A robótica exige que agentes tomem decisões contínuas e precisas em ambientes reais ou simulados. O RL tem permitido avanços notáveis na forma como robôs aprendem a se mover, interagir, manipular objetos e se adaptar a contextos diversos. Robôs móveis usam RL para aprender a evitar obstáculos, seguir rotas otimizadas e explorar ambientes desconhecidos. Robôs autônomos aprendem trajetos eficientes em fábricas, armazéns e ambientes domésticos. Um robô aspirador, por exemplo, aprende, com o tempo, quais rotas são mais eficientes para limpar o ambiente com menor consumo de bateria.

Robôs industriais treinam para pegar, girar e posicionar objetos com precisão. Aprendem a adaptar a força, o ângulo e a sequência de ações, mesmo com variações nos objetos. Braços robóticos em linhas de montagem aprendem a ajustar seus movimentos para encaixar peças com formas e posições variáveis, por exemplo.



Figura 31 – Robôs em uma indústria de vidros

Aprendizado simulado

Treinar robôs no mundo real pode ser caro, demorado e arriscado. Um erro no movimento, por exemplo, pode causar danos ao próprio robô ou ao ambiente. Por isso, utiliza-se o aprendizado simulado, em que o agente (robô virtual) aprende em um ambiente digital antes de ser testado na prática.

Simuladores como OpenAI Gym, MuJoCo e PyBullet oferecem ambientes virtuais que imitam com precisão as leis da física e a interação do robô com objetos. Nesses ambientes, o robô pode experimentar milhares de vezes, cometer erros, aprender com recompensas e melhorar seu comportamento – tudo isso sem qualquer risco físico ou custo de hardware.

Após o treinamento simulado, a política aprendida (ou seja, a estratégia de ação do agente) pode ser transferida para um robô físico real. Esse processo é conhecido como transferência de aprendizado (transfer learning).

Para que essa transferência funcione, é importante que o simulador seja realista o suficiente, com uma modelagem precisa da dinâmica física, sensores e atuadores do robô. Caso contrário, o que foi aprendido virtualmente pode não funcionar bem no mundo real – isso é chamado de lacuna de realidade, ou *gulf of reality*.

Imagine um braço robótico aprendendo a pegar uma caneca. Ele é treinado no simulador (por exemplo, MuJoCo), fazendo milhares de tentativas de pegar a caneca. A cada tentativa bem-sucedida, ele recebe uma recompensa. Com o tempo, ele aprende a sequência ideal de movimentos. Após o treinamento, o modelo aprendido é transferido para o braço robótico real, que agora pode executar a tarefa com segurança e eficiência.

O aprendizado simulado oferece diversas vantagens, como a redução de custos e a eliminação dos riscos de dano físico aos equipamentos durante o processo de treinamento. Além disso, permite a aceleração do tempo de aprendizado, já que milhares de simulações podem ser realizadas em um curto período. Outra vantagem importante é a possibilidade de realizar ajustes finos e testar uma ampla variedade de cenários e condições, o que contribui para o desenvolvimento de agentes mais robustos e adaptáveis antes de sua aplicação no mundo real.

Jogos são ambientes controlados e altamente interativos, ideais para treinar agentes com RL. Eles permitem avaliação objetiva de desempenho, diversidade de estratégias e exploração de estados complexos. Agentes de RL têm sido treinados para dominar jogos complexos como xadrez, Go e shogi. Utilizam redes neurais e busca em árvores de decisões para planejar jogadas com profundidade. Um exemplo famoso é o AlphaGo, da DeepMind, que derrotou o campeão mundial de Go ao aprender estratégias superiores às humanas, combinando RL e aprendizado profundo.

Agentes aprendem a jogar jogos de Atari, Mario, Doom, Dota 2 e outros a partir de pixels da tela e utilizam DQN para mapear imagens para ações ideais. Por exemplo: agentes da OpenAI aprenderam a jogar Dota 2 e venceram jogadores profissionais, desenvolvendo trabalho em equipe e táticas de ataque/defesa.



Observação

Vários agentes aprendem simultaneamente em ambientes colaborativos ou competitivos, como simulações de futebol, batalhas estratégicas e treinamentos cooperativos.

O aprendizado por reforço oferece diversas vantagens práticas em áreas como robótica e jogos. Uma das principais é o aprendizado por tentativa e erro, que permite aos agentes desenvolverem comportamentos otimizados com base na experiência adquirida. Além disso, o RL possibilita a adaptação a ambientes desconhecidos, permitindo que robôs e agentes operem de forma eficaz mesmo sem uma programação explícita para cada situação.

Outro benefício é a otimização de políticas a longo prazo, o que leva os agentes a aprenderem estratégias mais eficazes do que aquelas baseadas apenas em recompensas imediatas. Por fim, o RL favorece a tomada de decisão autônoma, reduzindo a necessidade de intervenção humana em tarefas complexas, como o controle de robôs ou a atuação em jogos dinâmicos.

Apesar de suas inúmeras vantagens, o uso do aprendizado por reforço também apresenta desafios significativos que precisam ser considerados. Em ambientes reais, especialmente na robótica, há limitações de tempo, sensores sujeitos a ruídos e um alto custo associado a erros, já que falhas podem danificar equipamentos físicos. Além disso, o processo de treinamento frequentemente requer milhares de interações com o ambiente, o que torna essencial o uso de simuladores para evitar riscos e acelerar o aprendizado.

Em jogos complexos ou cenários com múltiplas variáveis, o espaço de estados pode ser extremamente vasto, exigindo o uso de algoritmos eficientes e, muitas vezes, o apoio de redes neurais profundas para representar e generalizar o conhecimento de forma eficaz. Esses fatores evidenciam a necessidade de soluções técnicas robustas e recursos computacionais adequados para viabilizar o uso do RL em contextos práticos e de maior complexidade.

As aplicações do aprendizado por reforço em robótica e jogos mostram o enorme potencial dessa técnica para construir agentes capazes de aprender de forma autônoma, adaptar-se a novos cenários e tomar decisões inteligentes com base na experiência. Com o avanço de algoritmos e poder computacional, RL tem se tornado um pilar essencial na criação de IA com comportamento complexo, ultrapassando as limitações de abordagens tradicionais baseadas apenas em regras ou supervisão direta.



Resumo

Nesta unidade, exploramos os principais paradigmas de aprendizado de máquina utilizados em IA, apresentando de maneira estruturada os métodos de aprendizado supervisionado, não supervisionado e por reforço, com foco em suas características, algoritmos e aplicações práticas.

Compreendemos que o aprendizado supervisionado consiste no uso de dados rotulados para treinar modelos capazes de fazer previsões ou classificações. São discutidas as duas tarefas centrais desse paradigma: classificação, utilizada quando a saída é uma categoria (por exemplo, identificar se um e-mail é spam ou não), e regressão, empregada quando a saída é um valor numérico contínuo (como prever o preço de um imóvel). Apresentamos os principais algoritmos supervisionados, como árvores de decisão, regressão logística e máquinas de vetores de suporte (SVM), além de destacarmos aplicações reais, como sistemas de recomendação, análise preditiva e reconhecimento de padrões em imagens e textos.

Em seguida, aprofundamos nosso estudo em redes neurais artificiais, abordando sua estrutura em camadas, os neurônios artificiais e os pesos sinápticos que conectam essas unidades. Explicamos os mecanismos de ativação e propagação da informação, bem como o processo de aprendizado por retropropagação do erro, utilizado para ajustar os pesos da rede durante o treinamento.

Tratamos, depois, do aprendizado não supervisionado, no qual o modelo precisa encontrar estruturas ocultas em dados não rotulados. Focamos nossas análises nos algoritmos de agrupamento (clustering), especialmente o k-means, que busca dividir os dados em grupos com características semelhantes. Também discutimos outras abordagens de agrupamento, bem como aplicações práticas, como segmentação de mercado, análise de comportamento de usuários e detecção de anomalias.

Apresentamos o aprendizado por reforço, um paradigma baseado na interação entre um agente e um ambiente, em que o agente aprende a tomar decisões por meio de tentativa e erro, guiado por recompensas e penalidades. Introduzimos os principais elementos desse tipo de aprendizado, como estados, ações, recompensas, política e função de valor, além do conceito de MDP.

Em conjunto, esta unidade oferece uma base sólida para a compreensão dos diferentes tipos de aprendizado utilizados em IA, preparando o leitor para aplicar essas técnicas em contextos diversos e explorar soluções inteligentes para problemas reais.



Exercício

Questão 1. (FGV 2024, adaptada) Aprendizado de máquina (machine learning) é uma subárea da inteligência artificial que se concentra em desenvolver algoritmos e modelos que permitem que os sistemas aprendam e façam previsões ou tomem decisões baseadas em dados. O aprendizado de máquina pode ser dividido em aprendizado supervisionado, aprendizado não supervisionado e aprendizado por reforço.

No contexto de aprendizado de máquina supervisionado, das técnicas a seguir, a mais apropriada para lidar com um problema de regressão, no qual o objetivo é prever um valor numérico contínuo, é:

- A) a regressão linear.
- B) a regressão logística.
- C) o K-Nearest Neighbors (k-NN) para classificação.
- D) a Support Vector Machine (SVM) com kernel linear para classificação.
- E) a árvore de decisão para classificação.

Resposta correta: alternativa A.

Análise da questão

O enunciado da questão cita um problema de regressão no qual o objetivo é prever um valor contínuo. Esse tipo de problema pode ser encontrado em situações como preço de imóveis, temperatura, demanda de produtos etc.

Entre as técnicas listadas, a regressão linear é a mais apropriada, pois é projetada especificamente para prever valores numéricos contínuos. Ela funciona ajustando uma reta aos dados de treinamento, minimizando o erro entre os valores previstos e os valores reais por meio de métodos como mínimos quadrados. Sua simplicidade e sua eficiência a tornam uma ferramenta essencial em IA para problemas de regressão.

Questão 2. (Fundatec 2025, adaptada) A clusterização é uma técnica de aprendizado não supervisionado que agrupa dados com características similares.

Sobre as diferentes técnicas de clusterização, avalie as afirmativas a seguir.

I – Hierarchical clustering é mais adequado para grandes volumes de dados devido ao seu baixo custo computacional.

II – K-means é um algoritmo de clusterização que minimiza a variância dentro dos clusters ao calcular centros iterativamente.

III – DBSCAN é capaz de detectar clusters de formato arbitrário e de identificar pontos como ruído, se eles não pertencem a nenhum cluster.

É correto o que se afirma em:

A) I, apenas.

B) III, apenas.

C) I e II, apenas.

D) II e III, apenas.

E) I, II e III.

Resposta correta: alternativa D.

Análise das afirmativas

I – Afirmativa incorreta.

Justificativa: Hierarchical clustering tem alta complexidade computacional, o que o torna inadequado para grandes volumes de dados. Algoritmos como k-means são mais escaláveis.

II – Afirmativa correta.

Justificativa: k-means é um algoritmo que particiona os dados em clusters, minimiza iterativamente a variância intra-cluster e atualiza os centros até a convergência, ou seja, até que os centroides não mudem significativamente entre as iterações. O k-means é eficaz em grandes volumes de dados. Ele é conhecido por sua rapidez e por sua simplicidade de implementação.

III – Afirmativa correta.

Justificativa: o DBSCAN detecta clusters de formato arbitrário, não apenas esféricos, como o k-means. Ele identifica pontos de ruído, que não se encaixam em algum cluster. Além disso, o DBSCAN não requer o número de clusters como entrada, ao contrário do k-means.